iamSaikrishna143 all update ce579bf · 9 hours ago

5457 lines (4763 loc) · 193 KB

Code Blame Raw

```
// Your React app is getting slower when rendering a large list. How will you optimize i
// 1. List Virtualization (Windowing)
//  Don’t render all items at once—only render those visible in the viewport.
//  Use libraries:
// o    react-window (lightweight, modern)
//  react-virtualized (feature-rich)
// •    Example with react-window:
// •    import { FixedSizeList as List } from "react-window";
// •
// •    const MyList = ({ items }) => (
// •      <List
// •        height={400}
// •        itemCount={items.length}
// •        itemSize={35}
// •        width={300}
// •      >
// •        {({ index, style }) => (
// •          <div style={style}>{items[index]}</div>
// •        )}
// •      </List>
// •    );
// _____________________________________
// 🔑 2. Memoization
// •    Prevent unnecessary re-renders of list items.
// •    Use React.memo for item components:
// •    const ListItem = React.memo(({ item }) => {
// •      console.log("Rendering:", item.id);
// •      return <div>{item.name}</div>;
// •    });
// •    For callbacks, use useCallback:
// •    const handleClick = useCallback(() => { ... }, []);
// _____________________________________
// 🔑 3. Efficient Keys
// •    Always use stable, unique keys (id, not index).
// •    Prevents React from re-rendering items unnecessarily.
// _____________________________________
```

```
// 🔑 4. Pagination / Infinite Scroll
// •    If virtualization isn’t enough, load data in chunks (lazy loading).
// •    Example: Fetch 20–50 items at a time and append on scroll.
// ____________________________________________
// 🔑 5. Avoid Inline Functions & Objects in Render
// •    Inline objects/functions cause prop changes → re-renders.
// •    Instead:
// •    const style = { padding: 10 };
// •    return <div style={style}></div>;
// ____________________________________________
// 🔑 6. Selective Re-rendering with shouldComponentUpdate / React.memo
// •    Skip re-renders if props haven’t changed.
// •    Example with React.memo and custom comparison:
// •    const ListItem = React.memo(
// •      ({ item }) => <div>{item.name}</div>,
// •      (prev, next) => prev.item.id === next.item.id
// •    );
// ____________________________________________
// 🔑 7. Optimize State Management
// •    Avoid keeping the entire list in a parent state if only parts update.
// •    Push local state into items or use context selectively.
// ____________________________________________
// 🔑 8. Code Splitting / Lazy Loading
// •    If list items have heavy components (e.g., charts, images), lazy-load or use Rea


// How would you handle API call retries with exponential backoff in React?
// ✅ Implementation Approaches
// 1. Custom Utility Function
// Create a generic fetchWithRetry that can be reused in React apps.
// const fetchWithRetry = async (url, options = {}, retries = 3, delay = 1000) => {
//   try {
//     const response = await fetch(url, options);
//     if (!response.ok) throw new Error(`HTTP error! Status: ${response.status}`);
//     return response.json();
//   } catch (error) {
//     if (retries > 0) {
//       // exponential backoff with jitter
//       const backoff = delay * 2;
//       const jitter = Math.random() * 300;
//       await new Promise(res => setTimeout(res, backoff + jitter));
//       return fetchWithRetry(url, options, retries - 1, backoff);
//     }
//     throw error;
//   }
// };
// Usage inside a React component with useEffect:
// import { useEffect, useState } from "react";

// function UserList() {
//   const [users, setUsers] = useState([]);
//   const [error, setError] = useState(null);
```

```
//   useEffect(() => {
//     fetchWithRetry("https://api.example.com/users")
//       .then(setUsers)
//       .catch(setError);
//   }, []);

//   if (error) return <div>Error: {error.message}</div>;
//   return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
// }
// ____________________________________
// 2. Axios + Interceptors
// If you're using Axios, you can create a retry mechanism with interceptors.
// import axios from "axios";

// const api = axios.create({ baseURL: "https://api.example.com" });

// api.interceptors.response.use(null, async (error) => {
//   const { config } = error;
//   if (!config.__retryCount) config.__retryCount = 0;

//   if (config.__retryCount < 3) {
//     config.__retryCount += 1;
//     const backoff = Math.pow(2, config.__retryCount) * 1000;
//     await new Promise(res => setTimeout(res, backoff));
//     return api(config);
//   }

//   return Promise.reject(error);
// });
// Now just use api.get("/users") inside your React components.
// ____________________________________
// 3. React Query (TanStack Query) Built-in Retries
// If you're already using React Query, it has retries + exponential backoff built-in.
// Example:
// import { useQuery } from "@tanstack/react-query";
// import axios from "axios";

// function UserList() {
//   const { data, error, isLoading } = useQuery({
//     queryKey: ["users"],
//     queryFn: () => axios.get("/users").then(res => res.data),
//     retry: 3, // default is 3
//     retryDelay: attempt => Math.min(1000 * 2 ** attempt, 30000), // exponential backo
//   });

//   if (isLoading) return <p>Loading...</p>;
//   if (error) return <p>Error: {error.message}</p>;
//   return <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
// }
// ____________________________________
// ⚖️ Which Approach Should You Use?
// •    Custom function → lightweight apps, no external libs.
```

```
142  // •    Axios interceptors → if you already use Axios everywhere.
143  // •    React Query → best option for production apps (handles caching, retries, dedupli
144
145
146  // You have a component with heavy computations. How do you prevent unnecessary recalcul
147  // 🔑 Strategies
148  // 1. useMemo for Expensive Calculations
149  // •    Cache the result of a computation until dependencies change.
150  // •    Prevents recalculating on every render.
151  // import { useMemo, useState } from "react";
152
153  // function ExpensiveComponent({ numbers }) {
154  //   const [filter, setFilter] = useState("");
155
156  //   // expensive computation
157  //   const sortedNumbers = useMemo(() => {
158  //     console.log("Computing...");
159  //     return numbers.sort((a, b) => a - b);
160  //   }, [numbers]); // only re-runs when numbers change
161
162  //   const filtered = useMemo(() => {
163  //     return sortedNumbers.filter(n => n.toString().includes(filter));
164  //   }, [sortedNumbers, filter]);
165
166  //   return (
167  //     <>
168  //       <input value={filter} onChange={e => setFilter(e.target.value)} />
169  //       <ul>{filtered.map((n, i) => <li key={i}>{n}</li>)}</ul>
170  //     </>
171  //   );
172  // }
173  // ✅ Without useMemo, both sorting & filtering would run on every keystroke.
174  // ____________________________________________
175  // 2. useCallback for Stable Functions
176  // •    Heavy computations inside callbacks should also be memoized.
177  // •    Prevents creating new function references that cause re-renders in children.
178  // const handleCalculate = useCallback(() => {
179  //   return expensiveFunction();
180  // }, [dependencies]);
181  // ____________________________________________
182  // 3. Move Computations Outside Rendering
183  // •    If calculation does not depend on React state/props, move it outside the compone
184  // const precomputedData = heavyCalculation(); // runs once at module load
185
186  // function Component() {
187  //   return <div>{precomputedData}</div>;
188  // }
189  // ____________________________________________
190  // 4. Web Workers for Really Heavy Work
191  // •    For CPU-intensive tasks (e.g., image processing, large dataset crunching), move
192  // •    Keeps the UI thread responsive.
193  // You can use libraries like comlink or workerize.
```

```
// _____________________________________
// 5. Memoize Components (React.memo)
// •    If child components re-render unnecessarily because of props, wrap them with Rea
// •    Combined with useMemo for computations = big win.
// _____________________________________
// ✅ Rule of thumb:
// •    Use useMemo for expensive derived values.
// •    Use useCallback for stable functions.
// •    Use Web Workers if computation is so heavy it blocks UI.

// A child component re-renders even when props don’t change — what’s your debugging app

// 🔎 Step-by-step Debugging Approach
// 1. Confirm re-renders
// Add a render log:
// const Child = React.memo(({ value }) => {
//   console.log("Child re-render", value);
//   return <div>{value}</div>;
// });
// 👉 This tells you when and why React thinks it needs to render.
// _____________________________________
// 2. Check props identity
// Even if props look the same, objects, arrays, and functions create new references eve
// // ❌ Causes re-renders (new object every time)
// <Child data={{ id: 1 }} />

// // ✅ Fix with useMemo
// const memoizedData = useMemo(() => ({ id: 1 }), []);
// <Child data={memoizedData} />
// Same with callbacks:
// // ❌ New function each render
// <Child onClick={() => doSomething()} />

// // ✅ Stable reference
// const handleClick = useCallback(() => doSomething(), []);
// <Child onClick={handleClick} />
// _____________________________________
// 3. Wrap child in React.memo
// Ensure the child skips re-rendering when shallow props comparison shows no change.
// const Child = React.memo(function Child({ value }) {
//   return <div>{value}</div>;
// });
// If it still re-renders, props or context are changing.
// _____________________________________
// 4. Inspect parent re-renders
// If the parent re-renders (due to state/context), React will re-run the child too — ur
// _____________________________________
// 5. Look for context usage
// •    If the child consumes useContext, any context update re-renders all consumers.
// •    Fix: split contexts, or use selector-based contexts (e.g., use-context-selector)
// _____________________________________
// 6. Use tools
// •    React DevTools Profiler → see what re-renders and why.
```

```
// •    why-did-you-render → a library that tells you which props triggered a re-render.
// import React from "react";
// if (process.env.NODE_ENV === "development") {
//   const whyDidYouRender = require("@welldone-software/why-did-you-render");
//   whyDidYouRender(React, { trackAllPureComponents: true });
// }
// ____________________________________
// ✅ Quick checklist I run through:
// 1.   Add console logs → confirm re-render.
// 2.   Check for object/array/function props → memoize if needed.
// 3.   Wrap child with React.memo.
// 4.   Check parent/state/context triggers.
// 5.   Use Profiler or WDYR if still unclear.

//  How do you implement role-based authentication in a React app?
// Steps to Implement RBAC in React
// 1. Authentication & Roles Source
// •    Typically you get user info (with role(s)) after login via:
// o    JWT token with role claims.
// o    API response (e.g., { id: 1, name: "Sai", role: "admin" }).
// o    Auth provider (Firebase, Auth0, Keycloak, etc.).
// Store this info in:
// •    React Context
// •    Redux / Zustand
// •    Or directly in a global state hook.
// ____________________________________
// 2. Protect Routes with Role Checks
// Use React Router to guard routes.
// import { Navigate } from "react-router-dom";
// import { useAuth } from "./AuthContext";

// function ProtectedRoute({ children, allowedRoles }) {
//   const { user } = useAuth();

//   if (!user) {
//     return <Navigate to="/login" />;
//   }

//   if (!allowedRoles.includes(user.role)) {
//     return <Navigate to="/unauthorized" />;
//   }

//   return children;
// }
// Usage:
// <Route
//   path="/admin"
//   element={
//     <ProtectedRoute allowedRoles={['admin']}>
//       <AdminDashboard />
//     </ProtectedRoute>
//   }
```

```
// />
// _____________________________________
// 3. Role-based Component Rendering
// Sometimes you just need to hide/show parts of UI based on roles.
// function DeleteButton() {
//   const { user } = useAuth();
//   if (user.role !== "admin") return null;
//   return <button>Delete</button>;
// }
// Or with a helper HOC:
// const WithRole = ({ role, children }) => {
//   const { user } = useAuth();
//   return user?.role === role ? children : null;
// };
// _____________________________________
// 4. Store Tokens Securely
// •    Store JWT in httpOnly cookies (preferred) or localStorage (less secure).
// •    On each API request, send token → backend validates role before processing.
// _____________________________________
// 5. Backend Enforcement (Important!)
// ⚠️ React RBAC is only UI-level.
// Real security must be enforced server-side:
// •    API should check user’s role before returning protected data.
// •    Example (Express.js middleware):
// •    function authorizeRoles(...allowedRoles) {
// •      return (req, res, next) => {
// •        if (!allowedRoles.includes(req.user.role)) {
// •          return res.status(403).json({ message: "Forbidden" });
// •        }
// •        next();
// •      };
// •    }
// _____________________________________
// 6. Optional: Role Hierarchies & Permissions
// •    Instead of hardcoding if role === 'admin', define a permission map:
// const rolePermissions = {
//   admin: ["create", "edit", "delete"],
//   editor: ["create", "edit"],
//   user: ["view"]
// };
// Then check:
// function Can({ action, children }) {
//   const { user } = useAuth();
//   const permissions = rolePermissions[user.role] || [];
//   return permissions.includes(action) ? children : null;
// }
// Usage:
// <Can action="delete">
//   <DeleteButton />
// </Can>
// _____________________________________
// ✅ Summary
// 1.   Authenticate user → get roles (JWT, API, provider)
```

```
// 1.   Authenticate user → get roles (JWT, API, provider).
// 2.   Store roles in context/global state.
// 3.   Protect routes with ProtectedRoute.
// 4.   Show/hide UI with role checks.
// 5.   Enforce roles on backend for real security.

// You need to share state across deeply nested components. What options do you have?
// 🔑 Options for Sharing State Across Deeply Nested Components
// 1. Props Drilling (Basic but Not Scalable)
// •    Pass state from parent → child → grandchild → etc.
// •    Works for small apps, but quickly becomes painful.
// function Parent() {
//   const [theme, setTheme] = useState("light");
//   return <Child theme={theme} setTheme={setTheme} />;
// }
// ____________________________________________
// 2. React Context API (Most Common)
// •    Create a Context and wrap your tree.
// •    Any nested component can consume without prop drilling.
// const ThemeContext = createContext();

// function App() {
//   const [theme, setTheme] = useState("light");
//   return (
//     <ThemeContext.Provider value={{ theme, setTheme }}>
//       <DeepChild />
//     </ThemeContext.Provider>
//   );
// }

// function DeepChild() {
//   const { theme, setTheme } = useContext(ThemeContext);
//   return <button onClick={() => setTheme("dark")}>{theme}</button>;
// }
// ✅ Best for global app state like theme, auth, language.
// ____________________________________________
// 3. State Management Libraries
// If state gets complex or large, use a dedicated store:
// •    Redux Toolkit (most popular, structured, good for large apps).
// •    Zustand (lightweight, minimal boilerplate).
// •    Recoil / Jotai (atomic state management).
// •    MobX (reactive state).
// Example with Zustand:
// import create from "zustand";

// const useStore = create(set => ({
//   theme: "light",
//   setTheme: theme => set({ theme })
// }));

// function DeepChild() {
//   const { theme, setTheme } = useStore();
//   return <button onClick={() => setTheme("dark")}>{theme}</button>;
```

```
// }
// ____________________________________
// 4. URL / Router State
// •    Share state via query params or route params.
// •    Useful for filter/search state, so it persists on refresh or shareable links.
// <Route path="/dashboard/:tab" element={<Dashboard />} />
// ____________________________________
// 5. React Query (Server State)
// •    For data fetched from APIs, don’t lift state manually — use React Query (TanStac
// •    It handles caching, refetching, and sharing across components.
// const { data } = useQuery(["user"], fetchUser);
// ____________________________________
// 6. Event Emitters / Pub-Sub
// •    Useful for decoupled components that don’t share a parent.
// •    Libraries like mitt or even window.dispatchEvent can help.
// ____________________________________
// 7. Local Storage / Session Storage (Persistence)
// •    Store state in browser storage so it can be accessed across refreshes or tabs.
// •    Usually combined with context or store.
// ____________________________________
// ✅ Rule of Thumb
// •    Simple state, small app → props drilling.
// •    Medium app → Context API.
// •    Large/complex state → Redux, Zustand, Recoil, etc.
// •    Server state → React Query.
// •    Cross-tab or persistent state → LocalStorage / IndexedDB.

//
// How do you handle memory leaks in React apps (like setInterval, subscriptions)?
// 🔎 Common Sources of Memory Leaks in React
// 1.   setInterval / setTimeout → not cleared on unmount.
// 2.   Event listeners (window.addEventListener, DOM listeners).
// 3.   WebSocket / API subscriptions.
// 4.   Promises or async calls → updating state after unmount.
// 5.   Uncleaned refs or caches.
// ____________________________________
// ✅ Best Practices to Prevent Memory Leaks
// 1. Clean up side effects in useEffect
// Every useEffect can return a cleanup function:
// useEffect(() => {
//   const interval = setInterval(() => {
//     console.log("Running...");
//   }, 1000);

//   return () => clearInterval(interval); // ✅ cleanup
// }, []);
// For event listeners:
// useEffect(() => {
//   const handler = () => console.log("scrolling");
//   window.addEventListener("scroll", handler);

//   return () => window.removeEventListener("scroll", handler); // ✅ cleanup
```

```
// }, []);
// ____________________________________
// 2. Abort API Calls (fetch / Axios)
// If the component unmounts while a fetch is pending, calling setState afterward trigge
// useEffect(() => {
//   const controller = new AbortController();

//   fetch("/api/data", { signal: controller.signal })
//     .then(res => res.json())
//     .then(setData)
//     .catch(err => {
//       if (err.name !== "AbortError") console.error(err);
//     });

//   return () => controller.abort(); // ✅ cancel on unmount
// }, []);
// Axios has CancelToken or AbortController as well.
// ____________________________________
// 3. Guard State Updates After Unmount
// Sometimes you can track with a flag:
// useEffect(() => {
//   let isMounted = true;

//   asyncFunction().then(result => {
//     if (isMounted) setData(result);
//   });

//   return () => { isMounted = false; };
// }, []);
// ____________________________________
// 4. Cleanup WebSockets / Subscriptions
// useEffect(() => {
//   const socket = new WebSocket("ws://example.com");

//   socket.onmessage = (msg) => console.log(msg);

//   return () => socket.close(); // ✅ close on unmount
// }, []);
// ____________________________________
// 5. Use Libraries That Manage Cleanup
// •    React Query → automatically cancels queries if the component unmounts.
// •    RxJS + takeUntil → unsubscribes observables.
// •    zustand, jotai → handle state globally, avoiding dangling listeners.
// ____________________________________
// 6. React Dev Tools & Profiling
// •    Use Chrome DevTools Performance > Memory tab.
// •    Look for detached DOM nodes or growing memory usage after navigation.
// ____________________________________
// ⚖️ Rule of Thumb
// •    Always return a cleanup function from useEffect.
// •    Cancel intervals, timeouts, subscriptions, and fetch requests.
// •    Don’t update state if a component is unmounted.
// •    Use modern libraries (React Query, SWR) to handle async safely.
```

```
509
510 // How would you design a theme switcher (dark/light mode) in React?
511 // 🔑 Steps to Implement Theme Switcher
512 // 1. Decide Where to Store Theme State
513 // Options:
514 // •    React Context → so all components can access theme easily.
515 // •    CSS variables → for instant style changes.
516 // •    Local Storage → to remember user preference.
517 // •    System Preference (prefers-color-scheme) → use as a default.
518 // ____________________________________________
519 // 2. Create a Theme Context
520 // import { createContext, useContext, useState, useEffect } from "react";
521
522 // const ThemeContext = createContext();
523
524 // export const ThemeProvider = ({ children }) => {
525 //   // Load from localStorage or system preference
526 //   const getInitialTheme = () => {
527 //     if (localStorage.getItem("theme")) {
528 //       return localStorage.getItem("theme");
529 //     }
530 //     return window.matchMedia("(prefers-color-scheme: dark)").matches
531 //       ? "dark"
532 //       : "light";
533 //   };
534
535 //   const [theme, setTheme] = useState(getInitialTheme);
536
537 //   useEffect(() => {
538 //     localStorage.setItem("theme", theme);
539 //     document.documentElement.setAttribute("data-theme", theme); // hook for CSS varia
540 //   }, [theme]);
541
542 //   const toggleTheme = () =>
543 //     setTheme((prev) => (prev === "light" ? "dark" : "light"));
544
545 //   return (
546 //     <ThemeContext.Provider value={{ theme, toggleTheme }}>
547 //       {children}
548 //     </ThemeContext.Provider>
549 //   );
550 // };
551
552 // export const useTheme = () => useContext(ThemeContext);
553 // ____________________________________________
554 // 3. Hook CSS Variables into Theme
555 // In index.css (or Tailwind config):
556 // :root[data-theme="light"] {
557 //   --bg-color: #ffffff;
558 //   --text-color: #000000;
559 // }
560
```

```
// :root[data-theme="dark"] {
//   --bg-color: #121212;
//   --text-color: #ffffff;
// }

// body {
//   background: var(--bg-color);
//   color: var(--text-color);
// }
// ____________________________________________
// 4. Create a Toggle Button
// import { useTheme } from "./ThemeProvider";

// function ThemeToggle() {
//   const { theme, toggleTheme } = useTheme();

//   return (
//     <button onClick={toggleTheme}>
//       Switch to {theme === "light" ? "🌙 Dark" : "🌞 Light"} Mode
//     </button>
//   );
// }
// ____________________________________________
// 5. Wrap App with Provider
// import { ThemeProvider } from "./ThemeProvider";
// import ThemeToggle from "./ThemeToggle";

// function App() {
//   return (
//     <ThemeProvider>
//       <div>
//         <h1>Hello Theme Switcher</h1>
//         <ThemeToggle />
//       </div>
//     </ThemeProvider>
//   );
// }
// ____________________________________________
// ✨ Extras / Best Practices
// •    Animations → smooth transition with transition: background 0.3s;.
// •    Accessibility → keep good contrast for both modes.
// •    TailwindCSS users → enable darkMode: "class" in config and toggle a class (dark)
// •    Multiple themes → scale easily by adding more :root[data-theme="xyz"] sets.

// Your app needs offline support — how would you implement it?
// 🔑 Ways to Implement Offline Support
// 1. Service Workers (Caching)
// •    A service worker sits between your app and the network.
// •    It intercepts requests and serves cached responses when offline.
// •    Can pre-cache static assets (HTML, JS, CSS, icons) and runtime cache API calls.
// In React:
// •    If using CRA (Create React App) → it has built-in service worker setup (servicew
// •    If using Vite / Next.js → use Workbox or plugins
```

```
// •    If using Vite / Next.js → use Workbox or plugins.
// Example with Workbox (runtime caching):
// // service-worker.js
// import { registerRoute } from "workbox-routing";
// import { StaleWhileRevalidate } from "workbox-strategies";

// registerRoute(
//   ({ request }) => request.destination === "script" || request.destination === "style
//   new StaleWhileRevalidate()
// );
// ____________________________________________
// 2. Local Storage / IndexedDB
// For dynamic data (e.g., API responses, form inputs):
// •    Use localStorage for small key-value data.
// •    Use IndexedDB (via libraries like idb) for structured/offline-first storage.
// Example with IndexedDB:
// import { openDB } from 'idb';

// const db = await openDB('my-db', 1, {
//   upgrade(db) {
//     db.createObjectStore('users');
//   },
// });

// await db.put('users', { id: 1, name: "Sai" }, 1);
// const user = await db.get('users', 1);
// ____________________________________________
// 3. Background Sync (Optional but Powerful)
// •    Queue API requests when offline.
// •    Send them once the connection is back.
// •    Supported by service workers (workbox-background-sync).
// Example: saving a form submission while offline → sync when online.
// ____________________________________________
// 4. React Query / Apollo (Cache First Strategies)
// If you’re already using a data-fetching library:
// •    React Query → persists cache in localStorage/IndexedDB (persistQueryClient).
// •    Apollo Client → supports cache persistence for GraphQL.
// Example with React Query:
// import { persistQueryClient } from '@tanstack/react-query-persist-client';
// import { createSyncStoragePersister } from '@tanstack/query-sync-storage-persister';

// const persister = createSyncStoragePersister({ storage: window.localStorage });
// persistQueryClient({ queryClient, persister });
// ____________________________________________
// 5. Detect Offline Mode in React
// Show an offline banner or fallback UI:
// function useOnlineStatus() {
//   const [online, setOnline] = useState(navigator.onLine);

//   useEffect(() => {
//     const update = () => setOnline(navigator.onLine);
//     window.addEventListener("online", update);
//     window.addEventListener("offline", update);
```

```
666     //     return () => {
667     //       window.removeEventListener("online", update);
668     //       window.removeEventListener("offline", update);
669     //     };
670     //   }, []);
671     
672     //   return online;
673     // }
674     // Usage:
675     // const isOnline = useOnlineStatus();
676     // return !isOnline && <div>You are offline 🚫</div>;
677     // ____________________________________
678     // ✅ Full Offline Strategy
679     // 1.   Cache static assets → via service workers.
680     // 2.   Cache dynamic data → IndexedDB + React Query/Apollo persistence.
681     // 3.   Background sync → queue requests until online.
682     // 4.   Offline-aware UI → show status banners & graceful fallbacks.
683     
684     // What security features should be taken while designing API's?
685     // 🔑 1. Authentication
686     // •    Ensure that only authorized users can access your API.
687     // •    Common methods:
688     // o    JWT (JSON Web Tokens) → stateless, widely used for REST APIs.
689     // o    OAuth 2.0 / OpenID Connect → standard for third-party access.
690     // o    API Keys → simple but limited security; usually for service-to-service communica
691     // •    Always validate tokens and their expiration on the server side.
692     // ____________________________________
693     // 🔑 2. Authorization / Role-Based Access Control (RBAC)
694     // •    Once a user is authenticated, check what they’re allowed to do.
695     // •    Example:
696     // o    admin → can create/update/delete resources.
697     // o    user → can only read or update own data.
698     // •    Implement server-side checks for every request.
699     // ____________________________________
700     // 🔑 3. Input Validation & Sanitization
701     // •    Prevent injection attacks (SQL, NoSQL, command injection, XSS).
702     // •    Techniques:
703     // o    Validate all incoming data types and formats.
704     // o    Sanitize input strings (remove HTML or special characters if necessary).
705     // o    Use ORM/parameterized queries for databases (e.g., Prisma, Sequelize).
706     // ____________________________________
707     // 🔑 4. Rate Limiting & Throttling
708     // •    Protect your API from DDoS attacks or brute-force attempts.
709     // •    Limit number of requests per IP or per user:
710     // •    100 requests per minute per user/IP
711     // •    Use libraries like express-rate-limit or API gateway rate limits.
712     // ____________________________________
713     // 🔑 5. HTTPS / TLS
714     // •    Always serve API over HTTPS to encrypt data in transit.
715     // •    Never send sensitive info (tokens, passwords) over HTTP.
716     // ____________________________________
717     // 🔑 6. CORS (Cross-Origin Resource Sharing)
```

```
// •   Restrict which domains can access your API.
// •   Avoid Access-Control-Allow-Origin: * in production.
// •   Only allow trusted domains.
// ____________________________________
// 🔑 7. Data Encryption & Hashing
// •   Passwords → hash using strong algorithms (bcrypt, Argon2).
// •   Sensitive data → encrypt at rest (AES, etc.) if necessary.
// •   Never log sensitive info (passwords, tokens, credit card data).
// ____________________________________
// 🔑 8. Logging & Monitoring
// •   Track suspicious activities (failed logins, unusual requests).
// •   Monitor rate-limiting events and errors.
// •   Tools: ELK stack, Datadog, Prometheus, Sentry.
// ____________________________________
// 🔑 9. Versioning & Deprecation
// •   Keep multiple API versions and deprecate old ones gradually.
// •   This helps avoid breaking clients and potential security flaws in legacy endpoir
// ____________________________________
// 🔑 10. Prevent Common Attacks
// •   SQL/NoSQL Injection → use parameterized queries.
// •   XSS / CSRF → sanitize inputs, validate tokens.
// •   Mass Assignment → explicitly whitelist fields for updates.
// •   File Upload Vulnerabilities → validate file types, scan for malware.
// ____________________________________
// 🔑 11. Use Security Headers
// •   Content-Security-Policy → restrict scripts, images, styles.
// •   X-Frame-Options → prevent clickjacking.
// •   X-Content-Type-Options: nosniff → prevent MIME type sniffing.
// •   Strict-Transport-Security → enforce HTTPS.
// ____________________________________
// 🔑 12. API Gateway & Throttling (Optional)
// •   For larger apps, place API behind a gateway (e.g., AWS API Gateway, Kong, Nginx)
// •   Handles authentication, rate limiting, IP whitelisting, and logging centrally.
// ____________________________________
// ✅ Summary
// A secure API design includes:
// 1.   Authentication & role-based authorization.
// 2.   Input validation & sanitization.
// 3.   HTTPS / encrypted communication.
// 4.   Rate limiting & throttling.
// 5.   Logging, monitoring, and security headers.
// 6.   Safe handling of sensitive data.
// 7.   Protection against injection, XSS, CSRF, and file upload vulnerabilities.

// What is API throttling?
// 🔑 Key Points About API Throttling
// 1. Purpose
// •   Prevent overload: Stop a client from sending too many requests and crashing the
// •   Fair usage: Ensure all clients get a reasonable share of resources.
// •   Security: Mitigate DDoS or brute-force attacks.
// ____________________________________
// 2. How It Works
// •   The server sets a limit like:
```

```
771  // •    100 requests per minute per user/IP
772  // •    If a client exceeds the limit, the server rejects additional requests with a sta
773  // o    429 Too Many Requests
774  // •    Some APIs include headers like:
775  // •    X-RateLimit-Limit: 100
776  // •    X-RateLimit-Remaining: 20
777  // •    X-RateLimit-Reset: 60
778  // → Tells the client how many requests are left and when the limit resets.
779  // ____________________________________
780  // 3. Implementation Methods
781  // 1.   Fixed Window
782  // o    Count requests per fixed interval (e.g., per minute).
783  // o    Simple, but bursts at window edges can happen.
784  // 2.   Sliding Window
785  // o    Counts requests in a rolling time window.
786  // o    Smoother control than fixed window.
787  // 3.   Token Bucket
788  // o    Tokens are added to a bucket at a fixed rate.
789  // o    Each request consumes a token.
790  // o    Allows short bursts while controlling average rate.
791  // 4.   Leaky Bucket
792  // o    Requests flow into a “bucket” that leaks at a fixed rate.
793  // o    Controls burstiness by smoothing requests over time.
794  // ____________________________________
795  // 4. Common Use Cases
796  // •    Public APIs (Twitter, GitHub, Google Maps) to prevent abuse.
797  // •    Internal microservices to protect downstream services.
798  // •    Login endpoints to prevent brute-force attacks.
799  // ____________________________________
800  // 5. Difference Between Throttling & Rate Limiting
801  // Feature  Throttling  Rate Limiting
802  // Purpose  Smooth out bursts   Limit overall requests
803  // Behavior Delay or reject excess requests Reject requests exceeding limit
804  // Example  Allow 10 req/sec per user   Allow 100 req/min per API key
805  // ____________________________________
806  // 6. In React / Frontend Apps
807  // •    Frontend can implement client-side throttling to reduce unnecessary API calls:
808  // o    debounce for search input.
809  // o    throttle for scrolling or auto-save.
810  // ____________________________________
811  // In short: API throttling protects servers and ensures fair usage by limiting the rate
812
813  // How to improve the performance in React Application?
814  // 🔑 1. Optimize Rendering
815  // a. Use React.memo
816  // •    Prevent unnecessary re-renders of functional components when props haven’t chang
817  // const Child = React.memo(({ data }) => {
818  //   return <div>{data.name}</div>;
819  // });
820  // b. useMemo for Expensive Computations
821  // •    Memoize heavy calculations to avoid recomputation on every render.
822  // const sortedData = useMemo(() => data.sort((a, b) => a.value - b.value), [data]);
```

```
// c. useCallback for Stable Function References
// •    Prevent child components from re-rendering due to new function references.
// const handleClick = useCallback(() => console.log("Clicked"), []);
// ____________________________________________
// 🔑 2. Optimize Lists and Large Components
// a. Virtualization
// •    Render only visible items in large lists using libraries like:
// o    react-window
// o    react-virtualized
// import { FixedSizeList as List } from 'react-window';
// b. Pagination / Infinite Scroll
// •    Load data in chunks instead of rendering all items at once.
// ____________________________________________
// 🔑 3. Code Splitting & Lazy Loading
// •    Split bundles so users load only what's needed.
// const LazyComponent = React.lazy(() => import('./LazyComponent'));
// <Suspense fallback={<div>Loading...</div>}>
//   <LazyComponent />
// </Suspense>
// •    Use route-based splitting with React Router.
// ____________________________________________
// 🔑 4. Avoid Anonymous Objects/Functions in JSX
// •    Inline objects or functions create new references → re-renders.
// // ❌
// <MyComponent style={{ color: "red" }} />

// // ✅
// const style = { color: "red" };
// <MyComponent style={style} />
// ____________________________________________
// 🔑 5. Optimize Images & Assets
// •    Use optimized images (WebP, AVIF).
// •    Lazy load images:
// <img loading="lazy" src="image.jpg" alt="..." />
// •    Minify CSS/JS and use compression (gzip/brotli).
// ____________________________________________
// 🔑 6. Minimize State Updates
// •    Keep state local when possible.
// •    Avoid updating state in loops or unnecessarily.
// •    Use batching (React 18+ does automatic batching).
// ____________________________________________
// 🔑 7. Use Efficient State Management
// •    Heavy global state (Redux, Context) can trigger unnecessary re-renders.
// •    Split state into smaller slices or use Zustand/Recoil for more granular control.
// ____________________________________________
// 🔑 8. Web Performance Tools
// •    React DevTools Profiler → identify slow components.
// •    Chrome DevTools Lighthouse → audits app performance.
// •    Bundle Analyzer → find large dependencies (webpack-bundle-analyzer).
// ____________________________________________
// 🔑 9. Memoize Context Values
// •    Prevent all consumers from re-rendering by memoizing the value:
// const value = useMemo(() => ({ theme, toggleTheme }), [theme]);
```

```
// const value = useMemo(() => ({ theme, toggleTheme }), [theme]);
// <ThemeContext.Provider value={value}>
// ____________________________________________
// 🔑 10. Optimize Network Requests
// •    Debounce or throttle API calls (search input, scrolling).
// •    Cache responses using React Query / SWR.
// •    Prefetch data when possible.
// ____________________________________________
// ✅ Summary
// •    Component-level optimizations: React.memo, useMemo, useCallback.
// •    List & rendering optimizations: virtualization, pagination.
// •    Lazy loading & code splitting: React.lazy + Suspense.
// •    State & context management: granular state, memoized context.
// •    Assets & network: image optimization, caching, throttling.
// •    Monitoring: Profiler, Lighthouse, Bundle Analyzer.

// What is debouncing in React?
// Debouncing in React (or in JavaScript in general) is a technique used to limit how of
// It’s commonly used to improve performance and prevent unnecessary work, like API call
// ____________________________________________
// How it works:
// •    You wrap your function (e.g., an API call) in a debounce function.
// •    The function only executes after a certain period of inactivity.
// •    If the event keeps firing before the timeout ends, the timer resets.
// ____________________________________________
// Example in React:
// Suppose you want to fetch suggestions as the user types in a search box:
// import { useState, useEffect } from "react";

// function debounce(func, delay) {
//   let timer;
//   return function (...args) {
//     clearTimeout(timer);
//     timer = setTimeout(() => {
//       func(...args);
//     }, delay);
//   };
// }

// export default function Search() {
//   const [query, setQuery] = useState("");

//   const handleSearch = debounce((value) => {
//     console.log("Fetching API with:", value);
//     // Call your API here
//   }, 500); // wait 500ms after last keystroke

//   const handleChange = (e) => {
//     setQuery(e.target.value);
//     handleSearch(e.target.value);
//   };

//   return (
```

```
//     <input
//       type="text"
//       value={query}
//       onChange={handleChange}
//       placeholder="Search..."
//     />
//   );
// }
// ____________________________________
// Key Points:
// 1.   Prevents unnecessary executions – reduces API calls or heavy computations.
// 2.   Delay-based – executes only after the user stops triggering the event for a cert
// 3.   Often used with search inputs, window resizing, scroll events, or form validatic

// What if in a react app we need to develop a feature of auto save the inputs of a form
// For an auto-save form feature in a React app, debouncing is actually a perfect fit. T
// Here’s a step-by-step approach:
// ____________________________________
// 1. Setup the form state
// import { useState } from "react";

// function AutoSaveForm() {
//   const [formData, setFormData] = useState({
//     name: "",
//     email: "",
//   });

//   const handleChange = (e) => {
//     setFormData({ ...formData, [e.target.name]: e.target.value });
//   };

//   return (
//     <form>
//       <input
//         type="text"
//         name="name"
//         value={formData.name}
//         onChange={handleChange}
//         placeholder="Name"
//       />
//       <input
//         type="email"
//         name="email"
//         value={formData.email}
//         onChange={handleChange}
//         placeholder="Email"
//       />
//     </form>
//   );
// }

// export default AutoSaveForm;
```

```
// _____________________________________
// 2. Create a debounced save function
// You can use a custom debounce function or use a library like lodash.debounce.
// import { useEffect, useRef } from "react";

// function useDebounce(callback, delay) {
//   const timer = useRef();

//   const debouncedFunction = (...args) => {
//     if (timer.current) clearTimeout(timer.current);
//     timer.current = setTimeout(() => {
//       callback(...args);
//     }, delay);
//   };

//   return debouncedFunction;
// }
// _____________________________________
// 3. Integrate debounced auto-save
// function AutoSaveForm() {
//   const [formData, setFormData] = useState({ name: "", email: "" });

//   const saveData = (data) => {
//     console.log("Auto-saving data:", data);
//     // Make API call to save formData
//   };

//   const debouncedSave = useDebounce(saveData, 1000); // 1 second debounce

//   const handleChange = (e) => {
//     const newData = { ...formData, [e.target.name]: e.target.value };
//     setFormData(newData);
//     debouncedSave(newData); // auto-save after 1 second of inactivity
//   };

//   return (
//     <form>
//       <input
//         type="text"
//         name="name"
//         value={formData.name}
//         onChange={handleChange}
//         placeholder="Name"
//       />
//       <input
//         type="email"
//         name="email"
//         value={formData.email}
//         onChange={handleChange}
//         placeholder="Email"
//       />
//     </form>
//   );
```

```
// }
// ____________________________________
// ✅ Advantages of this approach:
// 1.   Auto-saves without interrupting user input.
// 2.   Reduces unnecessary API calls.
// 3.   Flexible – you can adjust the debounce delay.
// 4.   Works even if the user types slowly or pauses.

// Which react library is used to represent JSON data into charts, graphs, etc for bette
// 1. Recharts
// •    Website: https://recharts.org
// •    Pros:
// o    Simple, declarative, built on React components.
// o    Good for common charts: line, bar, area, pie, radar.
// o    Easy to use with JSON data.
// •    Cons:
// o    Less customizable for very complex charts.
// •    Example:
// import { LineChart, Line, XAxis, YAxis, Tooltip, CartesianGrid } from 'recharts';

// const data = [
//   { name: 'Jan', value: 30 },
//   { name: 'Feb', value: 45 },
//   { name: 'Mar', value: 60 },
// ];

// <LineChart width={400} height={300} data={data}>
//   <Line type="monotone" dataKey="value" stroke="#8884d8" />
//   <XAxis dataKey="name" />
//   <YAxis />
//   <Tooltip />
//   <CartesianGrid stroke="#ccc" />
// </LineChart>
// ____________________________________
// 2. Chart.js (with react-chartjs-2)
// •    Website: https://www.chartjs.org
// •    Pros:
// o    Extremely popular, mature, highly customizable.
// o    Supports many chart types and animations.
// •    Cons:
// o    Slightly heavier bundle size than Recharts.
// •    Example:
// import { Line } from 'react-chartjs-2';

// const data = {
//   labels: ['Jan', 'Feb', 'Mar'],
//   datasets: [
//     {
//       label: 'Sales',
//       data: [30, 45, 60],
//       borderColor: 'blue',
//       backgroundColor: 'lightblue',
```

```
//     },
//   ],
// };

// <Line data={data} />;
// ____________________________________________
// 3. Victory
// •    Website: https://formidable.com/open-source/victory
// •    Pros:
// o    Declarative, React-focused.
// o    Very flexible and supports complex data visualization.
// •    Cons:
// o    Learning curve is slightly higher than Recharts.
// ____________________________________________
// 4. Nivo
// •    Website: https://nivo.rocks
// •    Pros:
// o    Beautiful, responsive, interactive charts.
// o    Great for dashboards.
// o    Works directly with JSON data.
// •    Cons:
// o    Bundle size is bigger; might need tree-shaking for optimization.
// ____________________________________________
// Recommendation for React
// •    If you need simple and fast charts: Recharts ✅
// •    For advanced dashboards and interactive visuals: Nivo or Victory
// •    For highly customizable, production-ready charts: Chart.js with react-chartjs-2

// If you have to inform the backend developers about some API's are failing how will yo
// 1. Gather all relevant information
// Before notifying the backend team, make sure you have:
// •    API endpoint (e.g., POST /api/users/login)
// •    Request payload (if applicable)
// •    Response/error message from the API
// •    HTTP status code (e.g., 500, 404, 403)
// •    Steps to reproduce the issue
// •    Environment details (development, staging, or production)
// •    Time of occurrence (if it’s intermittent)
// ____________________________________________
// 2. Choose the communication channel
// Depending on your team’s workflow:
// •    Slack/Teams message – for quick notifications
// •    Email – if it requires detailed explanation
// •    Issue tracking system (Jira, Trello, GitHub Issues) – best for tracking and assi
// ____________________________________________
// 3. Structure your message
// Here’s a sample professional format for a Slack or Jira ticket:
// Title: API /api/users/login returning 500 Internal Server Error
// Description:
// Hi Team,

// The following API is failing:
```

```
1138  // - Endpoint: POST /api/users/login
1139  // - Environment: Staging
1140  // - Request Payload:
1141  //   {
1142  //     "email": "test@example.com",
1143  //     "password": "password123"
1144  //   }
1145  // - Response:
1146  //   Status: 500
1147  //   Message: "Internal Server Error"
1148  // - Steps to reproduce:
1149  //   1. Open the login form
1150  //   2. Enter valid credentials
1151  //   3. Click "Login"
1152  // - Time observed: 27-Sep-2025 10:30 AM IST
1153
1154  // Please investigate and advise if any changes are required on the frontend while the f
1155
1156  // Thanks,
1157  // [Your Name]
1158  // _____________________________________
1159  // 4. Optional: Add logs/screenshots
1160  // •    Include browser console errors or network tab screenshots.
1161  // •    Attach curl/Postman request screenshots to make it easier to debug.
1162  // _____________________________________
1163  // ✅ Key points:
1164  // •    Be specific (endpoint, payload, status code)
1165  // •    Be concise but include all necessary context
1166  // •    Be polite and collaborative – the goal is problem-solving, not blaming
1167
1168  // Which tool is used to improve code standards in react application to show warnings fo
1169  // 1. ESLint
1170  // •    What it is: A static code analysis tool for identifying problematic patterns or
1171  // •    Purpose:
1172  // o    Catch errors early (like undefined variables, incorrect hooks usage)
1173  // o    Enforce consistent coding style
1174  // o    Show warnings and errors directly in the IDE or console
1175  // •    Integration with React:
1176  // o    Use eslint-plugin-react for React-specific rules
1177  // o    Use eslint-plugin-jsx-a11y for accessibility checks
1178  // Example setup:
1179  // npm install eslint eslint-plugin-react eslint-plugin-jsx-a11y --save-dev
1180  // .eslintrc.json
1181  // {
1182  //   "env": {
1183  //     "browser": true,
1184  //     "es2021": true
1185  //   },
1186  //   "extends": [
1187  //     "eslint:recommended",
1188  //     "plugin:react/recommended"
1189  //   ],
```

```
1190    //   "plugins": ["react", "jsx-a11y"],
1191    //   "rules": {
1192    //     "react/prop-types": "off",
1193    //     "no-unused-vars": "warn"
1194    //   },
1195    //   "settings": {
1196    //     "react": {
1197    //       "version": "detect"
1198    //     }
1199    //   }
1200    // }
1201    // ____________________________________________
1202    // 2. Optional complementary tools
1203    // •    Prettier – Automatically formats code to a consistent style. Often used together
1204    // •    Husky + lint-staged – Run ESLint on staged files before commits to enforce code
1205    // ____________________________________________
1206    // ✅ How it helps in React
1207    // •    Warns about improper use of hooks (like missing dependencies in useEffect)
1208    // •    Highlights syntax errors or potential runtime errors
1209    // •    Enforces best practices across the team
1210    // •    Can be configured to show warnings or block builds for severe issues
1211
1212    // What are the unit testing tools used in React Application?
1213    // 1. Jest
1214    // •    Official site: https://jestjs.io/
1215    // •    Type: Test runner + assertion library
1216    // •    Pros:
1217    // o    Developed by Facebook; works out-of-the-box with React.
1218    // o    Supports mocking, snapshots, and coverage reports.
1219    // o    Fast and easy to integrate.
1220    // •    Usage Example:
1221    // import { sum } from './utils';
1222
1223    // test('adds 2 + 3 to equal 5', () => {
1224    //   expect(sum(2, 3)).toBe(5);
1225    // });
1226    // ____________________________________________
1227    // 2. React Testing Library (RTL)
1228    // •    Official site: https://testing-library.com/docs/react-testing-library/intro
1229    // •    Type: UI testing library
1230    // •    Pros:
1231    // o    Focuses on testing components from the user's perspective.
1232    // o    Encourages testing behavior rather than implementation details.
1233    // •    Usage Example:
1234    // import { render, screen, fireEvent } from '@testing-library/react';
1235    // import Button from './Button';
1236
1237    // test('button click calls handler', () => {
1238    //   const handleClick = jest.fn();
1239    //   render(<Button onClick={handleClick}>Click Me</Button>);
1240    //   fireEvent.click(screen.getByText(/click me/i));
1241    //   expect(handleClick).toHaveBeenCalledTimes(1);
```

```
// });
// _____________________________________
// 3. Enzyme (Legacy)
// •    Official site: https://enzymejs.github.io/enzyme/
// •    Type: Component testing utility
// •    Pros:
// o    Allows shallow rendering, full DOM rendering, and static rendering.
// o    Good for testing component internals.
// •    Cons:
// o    React Testing Library is preferred now for behavior-focused tests.
// _____________________________________
// 4. Other Supporting Tools
// •    MSW (Mock Service Worker): Mock API calls in tests
// •    Jest DOM: Adds DOM-specific assertions for React components
// •    Cypress (for E2E, not unit tests): Useful when unit + integration testing needs
// _____________________________________
// ✅ Recommended Stack for React Unit Testing
// •    Jest → test runner + assertions
// •    React Testing Library → component testing from user perspective
// •    Optional: Jest DOM + MSW → for DOM assertions and mocking APIs

// How do you handle API test case scenarios in React Application?
// 1. Identify API interactions
// •    Components using fetch or axios.
// •    Custom hooks for data fetching (useFetch, useQuery, etc.).
// •    Actions that trigger API calls (form submissions, button clicks).
// _____________________________________
// 2. Use a test runner and testing library
// •    Jest: for running tests and assertions.
// •    React Testing Library (RTL): for testing component behavior.
// _____________________________________
// 3. Mock API calls
// There are a few ways to mock APIs:
// a) Using Jest mocks
// For axios:
// // MyComponent.js
// import axios from "axios";
// import { useEffect, useState } from "react";

// export default function MyComponent() {
//   const [data, setData] = useState(null);
//   const [error, setError] = useState(null);

//   useEffect(() => {
//     axios.get("/api/users")
//       .then((res) => setData(res.data))
//       .catch((err) => setError("Failed to fetch"));
//   }, []);

//   if (error) return <div>{error}</div>;
//   if (!data) return <div>Loading...</div>;
//   return <div>{data.length} Users Loaded</div>;
// }
```

```
// Test with mocked axios:
// import { render, screen, waitFor } from "@testing-library/react";
// import MyComponent from "./MyComponent";
// import axios from "axios";

// jest.mock("axios");

// test("loads and displays users", async () => {
//   axios.get.mockResolvedValue({ data: [{ id: 1 }, { id: 2 }] });
//   render(<MyComponent />);
//   await waitFor(() => expect(screen.getByText("2 Users Loaded")).toBeInTheDocument())
// });

// test("handles API error", async () => {
//   axios.get.mockRejectedValue(new Error("Network Error"));
//   render(<MyComponent />);
//   await waitFor(() => expect(screen.getByText("Failed to fetch")).toBeInTheDocument()
// });
// ____________________________________________
// b) Using Mock Service Worker (MSW)
// •    MSW intercepts network requests in tests (or even dev) for realistic API mocking
// •    More scalable for complex API scenarios.
// Example:
// // handlers.js
// import { rest } from 'msw';

// export const handlers = [
//   rest.get('/api/users', (req, res, ctx) => {
//     return res(ctx.json([{ id: 1, name: "John" }]));
//   }),
// ];
// // test setup
// import { setupServer } from 'msw/node';
// import { handlers } from './handlers';
// const server = setupServer(...handlers);
// beforeAll(() => server.listen());
// afterEach(() => server.resetHandlers());
// afterAll(() => server.close());
// ____________________________________________
// 4. Handle different test scenarios
// 1.   Success scenario: API returns expected data → component renders correctly.
// 2.   Error scenario: API fails (network error, 500) → component shows error message.
// 3.   Empty data scenario: API returns empty array → check if component handles no-dat
// 4.   Loading state: Component shows “Loading…” while waiting for API.
// 5.   Edge cases: Unexpected data formats, partial data, or slow responses.
// ____________________________________________
// 5. Best Practices
// •    Do not call real APIs in unit tests → always mock.
// •    Test behavior, not implementation details → e.g., check what user sees, not inte
// •    Use waitFor or findBy in RTL for async API results.
// •    Combine unit and integration tests for full coverage.
```

```
// Give me a estimation of completing a auto save functionality with unit testing includ
// Here’s a practical estimation for implementing an auto-save form feature with unit te
// _____________________________________
// 1. Requirement Analysis & Planning – 1–2 hours
// •    Understand which fields need auto-save.
// •    Decide on debounce timing (e.g., 1–2 seconds after user stops typing).
// •    Decide whether API should be called per field change or the whole form.
// •    Plan for loading / saving / error states for UX feedback.
// _____________________________________
// 2. Implement Auto-Save Functionality – 4–6 hours
// •    Create form state using useState or useReducer.
// •    Implement debounce logic (custom hook or lodash debounce).
// •    Integrate API call for saving data (using fetch or axios).
// •    Add UI feedback: "Saving..." indicator and error messages.
// Deliverables:
// •    Functional form auto-saving after user stops typing.
// •    Visual feedback for users.
// _____________________________________
// 3. Unit Testing – 3–5 hours
// •    Tools: Jest + React Testing Library.
// •    Test cases to cover:
// 1.   Auto-save triggers after debounce timeout.
// 2.   API is called with correct payload.
// 3.   Error handling works correctly.
// 4.   Loading / saving states are rendered properly.
// 5.   Edge cases: empty fields, rapid input, network errors.
// •    Mock API calls using jest.mock() or MSW.
// Deliverables:
// •    All unit tests passing.
// •    Coverage for debounce, API calls, and UI states.
// _____________________________________
// 4. Integration & Manual Testing – 1–2 hours
// •    Test in development/staging environment.
// •    Validate auto-save behavior in different scenarios.
// •    Fix minor bugs or debounce timing adjustments.
// _____________________________________
// Estimated Total Time: 9–15 hours
// •    Low complexity form: ~9–10 hours.
// •    Medium complexity form (multiple fields + validations): ~12–15 hours.
// •    High complexity (dynamic fields, multi-step forms): 15–20+ hours.
// _____________________________________
// Tips to Speed Up
// 1.   Reuse a custom useDebounce or useAutoSave hook.
// 2.   Use mocked API calls in unit tests to avoid delays.
// 3.   Focus on behavior-driven tests rather than implementation details.
// 4.   Consider using Formik or React Hook Form to manage state efficiently.

//  Which cloud is used in your app development?
// In modern React (or full-stack) app development, several cloud providers are commonly
// _____________________________________
// 1. AWS (Amazon Web Services)
// •    Use Cases: Hosting, serverless backend (Lambda), databases (RDS, DynamoDB), stor
// •    Why it’s popular: Scalable, reliable, extensive ecosystem
```

```
// •    Why it's popular: Scalable, reliable, extensive ecosystem.
// •    Example:
// o    Hosting React frontend on S3 + CloudFront
// o    Backend APIs with API Gateway + Lambda
// o    Database: DynamoDB or RDS
// ____________________________________________
// 2. Firebase (by Google)
// •    Use Cases: Real-time database, hosting, authentication, serverless functions, ar
// •    Why it's popular: Easy setup, great for rapid development, especially for small-
// •    Example:
// o    Hosting React app on Firebase Hosting
// o    Auto-save or real-time data with Firestore
// o    Authentication with Firebase Auth
// ____________________________________________
// 3. Microsoft Azure
// •    Use Cases: Hosting, serverless functions, databases, DevOps pipelines.
// •    Example: React frontend on Azure Static Web Apps, backend APIs with Azure Functi
// ____________________________________________
// 4. Vercel / Netlify
// •    Use Cases: Hosting React/Next.js apps, serverless functions, continuous deployme
// •    Why it's popular: Super simple deployment with Git integration, automatic previe
// •    Example:
// o    Host a React SPA with CI/CD directly from GitHub.
// o    Use serverless functions for lightweight APIs.
// ____________________________________________
// 5. Heroku (less popular now)
// •    Easy deployment of full-stack apps.
// •    Good for prototyping and small projects.
// ____________________________________________
// Typical Stack for React Apps
// •    Frontend (React): Hosted on AWS S3 + CloudFront, Firebase Hosting, Vercel, or Ne
// •    Backend APIs: AWS Lambda, Node.js on EC2, or Firebase Functions.
// •    Database: DynamoDB, Firebase Firestore, PostgreSQL, MongoDB Atlas.
// •    CI/CD & Monitoring: GitHub Actions, AWS CodePipeline, or Vercel auto-deploy.

//
// How would you implement infinite scrolling in React?
// //  - Discuss how to manage state for pagination and loading indicators.
// //  - What are the different strategies (intersection observer vs scroll events)?

// . State Management for Infinite Scroll
// You typically need state to handle:
// 1.   Data items – the list of items fetched so far.
// 2.   Pagination info – current page number or cursor.
// 3.   Loading state – whether a fetch is in progress.
// 4.   Has more items – whether there's more data to fetch.
// Example with useState:
// import { useState, useEffect } from "react";

// const [items, setItems] = useState([]);
// const [page, setPage] = useState(1);
// const [loading, setLoading] = useState(false);
// const [hasMore, setHasMore] = useState(true);
```

```
// _____________________________________________
// 2. Fetching Data with Pagination
// Suppose the backend API supports page and limit parameters:
// const fetchItems = async () => {
//   if (loading || !hasMore) return;

//   setLoading(true);
//   try {
//     const res = await fetch(`/api/items?page=${page}&limit=20`);
//     const data = await res.json();

//     setItems(prev => [...prev, ...data.items]);
//     setHasMore(data.items.length > 0);
//     setPage(prev => prev + 1);
//   } catch (err) {
//     console.error(err);
//   } finally {
//     setLoading(false);
//   }
// };
// _____________________________________________
// 3. Displaying Loading Indicators
// You can show a loader at the bottom when fetching more items:
// {loading && <div>Loading more items...</div>}
// _____________________________________________
// 4. Strategies for Detecting Scroll Position
// There are two main strategies:
// A. Scroll Event Listener
// •    Attach onScroll to the container or window.
// •    Calculate if the user is near the bottom, then fetch more.
// useEffect(() => {
//   const handleScroll = () => {
//     if (
//       window.innerHeight + document.documentElement.scrollTop + 100 >=
//       document.documentElement.offsetHeight
//     ) {
//       fetchItems();
//     }
//   };

//   window.addEventListener("scroll", handleScroll);
//   return () => window.removeEventListener("scroll", handleScroll);
// }, [loading, hasMore]);
// Pros: Works everywhere, easy to understand.
// Cons: Can be less efficient because scroll events fire frequently. May require thrott
// _____________________________________________
// B. Intersection Observer (Recommended)
// •    Attach a ref to a sentinel element at the bottom.
// •    When the sentinel is visible, load more items.
// import { useRef, useEffect } from "react";

// const loaderRef = useRef(null);
```

```
1504
1505  // useEffect(() => {
1506  //   const observer = new IntersectionObserver(
1507  //     entries => {
1508  //       if (entries[0].isIntersecting && hasMore) {
1509  //         fetchItems();
1510  //       }
1511  //     },
1512  //     { root: null, rootMargin: "100px", threshold: 0 }
1513  //   );
1514
1515  //   if (loaderRef.current) observer.observe(loaderRef.current);
1516
1517  //   return () => {
1518  //     if (loaderRef.current) observer.unobserve(loaderRef.current);
1519  //   };
1520  // }, [hasMore, loading]);
1521  // Then, render:
1522  // <div ref={loaderRef}></div>
1523  // Pros: Efficient, doesn’t require manual scroll calculations, works well with variable
1524  // Cons: Slightly more complex than scroll events.
1525  // ____________________________________
1526  // 5. Putting It Together
1527  // function InfiniteScrollList() {
1528  //   const [items, setItems] = useState([]);
1529  //   const [page, setPage] = useState(1);
1530  //   const [loading, setLoading] = useState(false);
1531  //   const [hasMore, setHasMore] = useState(true);
1532  //   const loaderRef = useRef(null);
1533
1534  //   const fetchItems = async () => {
1535  //     if (loading || !hasMore) return;
1536  //     setLoading(true);
1537  //     try {
1538  //       const res = await fetch(`/api/items?page=${page}&limit=20`);
1539  //       const data = await res.json();
1540  //       setItems(prev => [...prev, ...data.items]);
1541  //       setHasMore(data.items.length > 0);
1542  //       setPage(prev => prev + 1);
1543  //     } finally {
1544  //       setLoading(false);
1545  //     }
1546  //   };
1547
1548  //   useEffect(() => {
1549  //     const observer = new IntersectionObserver(
1550  //       entries => {
1551  //         if (entries[0].isIntersecting) fetchItems();
1552  //       },
1553  //       { root: null, rootMargin: "100px" }
1554  //     );
1555
1556  //     if (loaderRef.current) observer.observe(loaderRef.current);
```

```
//     return () => loaderRef.current && observer.unobserve(loaderRef.current);
//   }, [loaderRef.current]);

//   return (
//     <div>
//       {items.map(item => (
//         <div key={item.id}>{item.name}</div>
//       ))}
//       {loading && <p>Loading...</p>}
//       <div ref={loaderRef} />
//     </div>
//   );
// }
// ____________________________________
// ✅ Summary of Key Points
// •    State: Track items, page, loading, and hasMore.
// •    Scroll Detection: Either scroll events or intersection observer.
// •    Loading UI: Show loader when fetching.
// •    Efficiency: Intersection Observer is preferred for better performance.

// Design and implement a debounced search input component.
// //  - How would you handle race conditions in API responses?
// //  - What about caching previous search results?

// 1. Debounced Search Input Component
// Debouncing ensures that API calls are not triggered on every keystroke but only after
// import { useState, useEffect, useRef } from "react";

// function DebouncedSearch({ searchApi, delay = 500 }) {
//   const [query, setQuery] = useState("");
//   const [results, setResults] = useState([]);
//   const [loading, setLoading] = useState(false);

//   // Cache to store previous search results
//   const cache = useRef({});

//   // Ref to handle race conditions
//   const latestRequest = useRef(0);

//   useEffect(() => {
//     if (!query) {
//       setResults([]);
//       return;
//     }

//     // Check cache first
//     if (cache.current[query]) {
//       setResults(cache.current[query]);
//       return;
//     }

//     const currentRequestId = ++latestRequest.current;
```

```
1609  //     setLoading(true);
1610
1611  //     const handler = setTimeout(async () => {
1612  //       try {
1613  //         const data = await searchApi(query);
1614
1615  //         // Only set results if this is the latest request
1616  //         if (currentRequestId === latestRequest.current) {
1617  //           setResults(data);
1618  //           cache.current[query] = data; // cache results
1619  //         }
1620  //       } catch (err) {
1621  //         console.error(err);
1622  //       } finally {
1623  //         if (currentRequestId === latestRequest.current) {
1624  //           setLoading(false);
1625  //         }
1626  //       }
1627  //     }, delay);
1628
1629  //     return () => clearTimeout(handler);
1630  //   }, [query, searchApi, delay]);
1631
1632  //   return (
1633  //     <div>
1634  //       <input
1635  //         type="text"
1636  //         value={query}
1637  //         onChange={(e) => setQuery(e.target.value)}
1638  //         placeholder="Search..."
1639  //       />
1640  //       {loading && <p>Loading...</p>}
1641  //       <ul>
1642  //         {results.map((item) => (
1643  //           <li key={item.id}>{item.name}</li>
1644  //         ))}
1645  //       </ul>
1646  //     </div>
1647  //   );
1648  // }
1649
1650  // export default DebouncedSearch;
1651  // ____________________________________________
1652  // 2. Handling Race Conditions
1653  // Race conditions occur when multiple API calls are in flight, and an earlier request r
1654  // •    Use a latestRequest counter (useRef) that increments with every new API call.
1655  // •    Only update results if the response matches the latest request ID.
1656  // const currentRequestId = ++latestRequest.current;
1657  // if (currentRequestId === latestRequest.current) {
1658  //   setResults(data);
1659  // }
1660  // This ensures stale responses are ignored.
1661  //
```

```
// _____________________________________________
// 3. Caching Previous Search Results
// •    Use a useRef object (cache) to store results keyed by the search query.
// •    On a repeated query, return the cached data instead of calling the API.
// if (cache.current[query]) {
//   setResults(cache.current[query]);
//   return;
// }
// This improves performance and reduces unnecessary API calls.
// _____________________________________________
// ✅ Key Points
// 1.   Debounce API calls to reduce network requests.
// 2.   Handle race conditions with a request counter or AbortController.
// 3.   Cache previous results to improve UX and reduce load on the server.
// 4.   The component is reusable and works for any API you pass as searchApi.

// How would you implement a virtualized list component?
// //  - Discuss handling dynamic item heights vs fixed heights.
// //  - What about scroll position maintenance during updates?
// 1. Why Virtualization?
// Rendering thousands of items in the DOM can be slow and memory-intensive. Virtualizat
// _____________________________________________
// 2. State & Core Concepts
// We need to track:
// •    scrollTop – current scroll position.
// •    containerHeight – visible viewport height.
// •    startIndex & endIndex – the range of items to render.
// •    itemHeights (optional) – for dynamic heights.
// const [scrollTop, setScrollTop] = useState(0);
// const containerRef = useRef(null);
// _____________________________________________
// 3. Fixed Height Items (Simplest Case)
// If each item has a fixed height, calculations are simple:
// const ITEM_HEIGHT = 50; // px
// const buffer = 5;

// const startIndex = Math.floor(scrollTop / ITEM_HEIGHT);
// const endIndex = Math.min(
//   items.length - 1,
//   Math.ceil((scrollTop + containerHeight) / ITEM_HEIGHT) + buffer
// );

// const visibleItems = items.slice(startIndex, endIndex + 1);

// const offsetY = startIndex * ITEM_HEIGHT;
// Render the visible items with a spacer to maintain scroll height:
// <div
//   ref={containerRef}
//   style={{ overflowY: "auto", height: containerHeight }}
//   onScroll={(e) => setScrollTop(e.target.scrollTop)}
// >
//   <div style={{ height: items.length * ITEM_HEIGHT, position: "relative" }}>
//     {visibleItems.map((item, index) => (
```

```
1714    //       <div
1715    //         key={item.id}
1716    //         style={{
1717    //           position: "absolute",
1718    //           top: offsetY + index * ITEM_HEIGHT,
1719    //           height: ITEM_HEIGHT,
1720    //           width: "100%",
1721    //         }}
1722    //       >
1723    //         {item.content}
1724    //       </div>
1725    //     ))}
1726    //   </div>
1727    // </div>
1728    // ✅ Pros: Simple and very fast.
1729    // ❌ Cons: Only works if item heights are consistent.
1730    // ____________________________________________
1731    // 4. Dynamic Item Heights
1732    // For dynamic heights:
1733    // 1.   Track heights for each item in a map:
1734    // const itemHeights = useRef({}); // id -> height
1735    // 2.   Measure rendered items with ref or ResizeObserver:
1736    // <div
1737    //   ref={(el) => {
1738    //     if (el) itemHeights.current[item.id] = el.getBoundingClientRect().height;
1739    //   }}
1740    // >
1741    //   {item.content}
1742    // </div>
1743    // 3.   Compute cumulative offsets:
1744    // const offsets = items.map((item, i) =>
1745    //   i === 0 ? 0 : offsets[i - 1] + (itemHeights.current[items[i - 1].id] || defaultHeig
1746    // );
1747    // 4.   Render visible items based on scroll position and offsets:
1748    // const startIndex = offsets.findIndex(offset => offset >= scrollTop) - 1;
1749    // const endIndex = offsets.findIndex(offset => offset >= scrollTop + containerHeight);
1750    // ✅ Pros: Works with variable height items.
1751    // ❌ Cons: More complex and slower than fixed height; measuring heights dynamically ad
1752    // ____________________________________________
1753    // 5. Maintaining Scroll Position During Updates
1754    // When items are added, removed, or resized:
1755    // •    Use scrollTop + cumulative height calculations to prevent “jumping”.
1756    // •    If prepending items, adjust scrollTop so the viewport stays consistent:
1757    // containerRef.current.scrollTop += addedItemsHeight;
1758    // •    Libraries like react-virtualized or react-window handle this automatically for m
1759    // ____________________________________________
1760    // 6. Example Using react-window (Recommended)
1761    // For fixed height items:
1762    // import { FixedSizeList as List } from "react-window";
1763
1764    // <List
1765    //   height={500}
```

```
1766    //   itemCount={items.length}
1767    //   itemSize={50}
1768    //   width="100%"
1769    // >
1770    //   {({ index, style }) => <div style={style}>{items[index].content}</div>}
1771    // </List>
1772    // For variable height items, use VariableSizeList with itemSize(index) returning height
1773    // ____________________________________________
1774    // ✅ Key Takeaways
1775    // 1.   Fixed height items: simplest and fastest, calculate start/end indices via scroll
1776    // 2.   Dynamic height items: track individual heights, compute offsets, measure element
1777    // 3.   Scroll maintenance: adjust scrollTop when list changes to prevent jumps.
1778    // 4.   Use libraries (react-window, react-virtualized) for production-ready virtualizat
1779
1780    // Create a custom hook for data fetching with caching.
1781    // //  - How would you handle request deduplication?
1782    // //  - What about background refetching and optimistic updates?
1783
1784    // 1. Hook Requirements
1785    // The hook should handle:
1786    // 1.   Caching – reuse previous results for the same key.
1787    // 2.   Request deduplication – avoid multiple identical requests simultaneously.
1788    // 3.   Background refetching – fetch fresh data in the background without blocking UI.
1789    // 4.   Optimistic updates – temporarily update data before a server confirms success.
1790    // 5.   Loading & error states – standard UX handling.
1791    // ____________________________________________
1792    // 2. Implementation: useFetch
1793    // import { useState, useEffect, useRef } from "react";
1794
1795    // const cache = new Map();
1796    // const ongoingRequests = new Map();
1797
1798    // function useFetch(key, fetcher, options = {}) {
1799    //   const { initialData = null, refetchInterval = 0 } = options;
1800
1801    //   const [data, setData] = useState(cache.get(key) || initialData);
1802    //   const [loading, setLoading] = useState(!cache.has(key));
1803    //   const [error, setError] = useState(null);
1804
1805    //   const isMounted = useRef(true);
1806
1807    //   useEffect(() => {
1808    //     return () => { isMounted.current = false; };
1809    //   }, []);
1810
1811    //   const fetchData = async () => {
1812    //     // Deduplication: return existing promise if request is ongoing
1813    //     if (ongoingRequests.has(key)) return ongoingRequests.get(key);
1814
1815    //     setLoading(true);
1816    //     const promise = fetcher()
1817    //       .then((res) => {
1818    //         cache.set(key, res);
```

```
1819    //         if (isMounted.current) setData(res);
1820    //         return res;
1821    //       })
1822    //       .catch((err) => {
1823    //         if (isMounted.current) setError(err);
1824    //         throw err;
1825    //       })
1826    //       .finally(() => {
1827    //         ongoingRequests.delete(key);
1828    //         if (isMounted.current) setLoading(false);
1829    //       });
1830
1831    //     ongoingRequests.set(key, promise);
1832    //     return promise;
1833    //   };
1834
1835    //   useEffect(() => {
1836    //     fetchData();
1837
1838    //     // Background refetching
1839    //     let interval;
1840    //     if (refetchInterval > 0) {
1841    //       interval = setInterval(() => {
1842    //         fetchData();
1843    //       }, refetchInterval);
1844    //     }
1845
1846    //     return () => interval && clearInterval(interval);
1847    //   }, [key]);
1848
1849    //   // Optimistic update helper
1850    //   const mutate = (updater) => {
1851    //     setData((prev) => {
1852    //       const newData = typeof updater === "function" ? updater(prev) : updater;
1853    //       cache.set(key, newData);
1854    //       return newData;
1855    //     });
1856    //   };
1857
1858    //   return { data, loading, error, refetch: fetchData, mutate };
1859    // }
1860
1861    // export default useFetch;
1862    // ____________________________________
1863    // 3. Usage Example
1864    // function App() {
1865    //   const { data, loading, error, refetch, mutate } = useFetch(
1866    //     "todos",
1867    //     () => fetch("/api/todos").then(res => res.json()),
1868    //     { refetchInterval: 10000 } // refetch every 10s
1869    //   );
1870
```

```
1871  //   const addTodo = async (todo) => {
1872  //     // Optimistic update
1873  //     mutate(prev => [...prev, todo]);
1874  //
1875  //     try {
1876  //       await fetch("/api/todos", {
1877  //         method: "POST",
1878  //         headers: { "Content-Type": "application/json" },
1879  //         body: JSON.stringify(todo)
1880  //       });
1881  //       // background refetch ensures data is synced
1882  //     } catch (err) {
1883  //       // rollback if API fails
1884  //       mutate(prev => prev.filter(t => t !== todo));
1885  //     }
1886  //   };
1887  //
1888  //   if (loading) return <p>Loading...</p>;
1889  //   if (error) return <p>Error: {error.message}</p>;
1890  //
1891  //   return (
1892  //     <div>
1893  //       <ul>
1894  //         {data.map(todo => <li key={todo.id}>{todo.title}</li>)}
1895  //       </ul>
1896  //       <button onClick={() => addTodo({ id: Date.now(), title: "New Todo" })}>
1897  //         Add Todo
1898  //       </button>
1899  //       <button onClick={refetch}>Refetch</button>
1900  //     </div>
1901  //   );
1902  // }
1903  // ____________________________________
1904  // 4. Key Concepts Explained
1905  // Feature  Implementation
1906  // Caching  Map stores results keyed by key to avoid refetching.
1907  // Request deduplication    ongoingRequests map prevents multiple identical fetches simu
1908  // Background refetching    Optional refetchInterval triggers periodic updates without b
1909  // Optimistic updates   mutate allows temporary UI updates before server confirms. Can r
1910  // Loading/Error handling   Standard loading and error states exposed to component.
1911  // ____________________________________
1912  // ✅ Advantages:
1913  // •    Fully reusable and composable.
1914  // •    Reduces network calls with caching and deduplication.
1915  // •    Supports reactive UI with optimistic updates.
1916  // •    Can easily integrate with APIs, GraphQL, or REST.
1917
1918  // Implement a modal component with proper accessibility.
1919  // //  - Discuss backdrop click handling and escape key functionality.
1920  // //  - How would you prevent body scroll when modal is open?
1921
1922  // 1. Accessibility Considerations
1923  // For an accessible modal:
```

```
// For an accessible modal:
// •    Use role="dialog" and aria-modal="true".
// •    Focus should move to the modal when opened.
// •    Focus should be trapped inside the modal while open.
// •    Return focus to the previously focused element when closed.
// ____________________________________________
// 2. Modal Component Implementation
// import { useEffect, useRef } from "react";

// function Modal({ isOpen, onClose, children }) {
//   const modalRef = useRef(null);
//   const previousActiveElement = useRef(null);

//   useEffect(() => {
//     if (!isOpen) return;

//     // Save previously focused element
//     previousActiveElement.current = document.activeElement;

//     // Move focus to modal
//     modalRef.current?.focus();

//     // Prevent body scroll
//     const originalOverflow = document.body.style.overflow;
//     document.body.style.overflow = "hidden";

//     // Escape key handling
//     const handleKeyDown = (e) => {
//       if (e.key === "Escape") onClose();
//     };
//     document.addEventListener("keydown", handleKeyDown);

//     return () => {
//       document.body.style.overflow = originalOverflow;
//       document.removeEventListener("keydown", handleKeyDown);
//       // Restore focus to previously active element
//       previousActiveElement.current?.focus();
//     };
//   }, [isOpen, onClose]);

//   if (!isOpen) return null;

//   const handleBackdropClick = (e) => {
//     if (e.target === e.currentTarget) {
//       onClose();
//     }
//   };

//   return (
//     <div
//       onClick={handleBackdropClick}
//       style={{
//         position: "fixed",
```

```
1976  //         inset: 0,
1977  //         background: "rgba(0,0,0,0.5)",
1978  //         display: "flex",
1979  //         justifyContent: "center",
1980  //         alignItems: "center",
1981  //         zIndex: 1000,
1982  //       }}
1983  //     >
1984  //       <div
1985  //         ref={modalRef}
1986  //         tabIndex={-1}
1987  //         role="dialog"
1988  //         aria-modal="true"
1989  //         style={{
1990  //           background: "#fff",
1991  //           padding: "1.5rem",
1992  //           borderRadius: "8px",
1993  //           maxWidth: "500px",
1994  //           width: "90%",
1995  //           outline: "none",
1996  //         }}
1997  //       >
1998  //         {children}
1999  //         <button onClick={onClose} style={{ marginTop: "1rem" }}>
2000  //           Close
2001  //         </button>
2002  //       </div>
2003  //     </div>
2004  //   );
2005  // }
2006
2007  // export default Modal;
2008  // ____________________________________
2009  // 3. Key Features Explained
2010  // Feature  Implementation
2011  // Backdrop click   onClick on the backdrop div, check e.target === e.currentTarget to a
2012  // Escape key   Add keydown listener for Escape and call onClose.
2013  // Body scroll lock Set document.body.style.overflow = "hidden" when modal opens, restor
2014  // Focus management Save previous focus, focus modal on open, restore focus on close.
2015  // ARIA attributes  role="dialog" and aria-modal="true" for screen readers.
2016  // ____________________________________
2017  // 4. Usage Example
2018  // import { useState } from "react";
2019  // import Modal from "./Modal";
2020
2021  // function App() {
2022  //   const [isOpen, setIsOpen] = useState(false);
2023
2024  //   return (
2025  //     <div>
2026  //       <button onClick={() => setIsOpen(true)}>Open Modal</button>
2027  //       <Modal isOpen={isOpen} onClose={() => setIsOpen(false)}>
```

```
2028    //         <h2>Accessible Modal</h2>
2029    //         <p>This modal traps focus and handles escape/backdrop click.</p>
2030    //       </Modal>
2031    //     </div>
2032    //   );
2033    // }
2034    // ____________________________________________
2035    // 5. Additional Enhancements (Optional)
2036    // 1.   Focus Trap: Use a library like focus-trap-react for robust focus management.
2037    // 2.   Animations: Use framer-motion or CSS transitions for smooth open/close.
2038    // 3.   Multiple Modals: Keep a stack of open modals and restore body scroll only when a
2039    // 4.   Keyboard Navigation: Ensure tabbing cycles within modal content.
2040    // ____________________________________________
2041    // This implementation ensures accessibility, proper scroll management, and intuitive ba
2042
2043    //  6. Design a global state management solution without external libraries.
2044    // //  - How would you implement a Context + useReducer pattern?
2045    // //  - How would you handle middleware for logging or async actions?
2046
2047    // 1. Core Idea: Context + useReducer
2048    // •    Context provides a global provider for the state.
2049    // •    useReducer manages state updates with a centralized reducer.
2050    // •    Dispatch function can be extended with middleware.
2051    // ____________________________________________
2052    // 2. Step 1: Create the Reducer
2053    // const initialState = {
2054    //   user: null,
2055    //   todos: [],
2056    //   loading: false,
2057    //   error: null,
2058    // };
2059
2060    // function reducer(state, action) {
2061    //   switch (action.type) {
2062    //     case "SET_USER":
2063    //       return { ...state, user: action.payload };
2064    //     case "ADD_TODO":
2065    //       return { ...state, todos: [...state.todos, action.payload] };
2066    //     case "SET_LOADING":
2067    //       return { ...state, loading: action.payload };
2068    //     case "SET_ERROR":
2069    //       return { ...state, error: action.payload };
2070    //     default:
2071    //       return state;
2072    //   }
2073    // }
2074    // ____________________________________________
2075    // 3. Step 2: Create Context
2076    // import { createContext, useReducer, useContext } from "react";
2077
2078    // const GlobalStateContext = createContext();
2079    // const GlobalDispatchContext = createContext();
2080
```

```
// export function useGlobalState() {
//   return useContext(GlobalStateContext);
// }

// export function useGlobalDispatch() {
//   return useContext(GlobalDispatchContext);
// }
// ____________________________________________
// 4. Step 3: Provider with Middleware Support
// We can enhance the dispatch function to handle logging and async actions.
// export function GlobalProvider({ children }) {
//   const [state, baseDispatch] = useReducer(reducer, initialState);

//   // Middleware-enhanced dispatch
//   const dispatch = (action) => {
//     if (typeof action === "function") {
//       // Async action
//       return action(dispatch, () => state);
//     }

//     // Logging middleware
//     console.log("[Dispatching]", action);
//     baseDispatch(action);
//   };

//   return (
//     <GlobalStateContext.Provider value={state}>
//       <GlobalDispatchContext.Provider value={dispatch}>
//         {children}
//       </GlobalDispatchContext.Provider>
//     </GlobalStateContext.Provider>
//   );
// }
// ____________________________________________
// 5. Step 4: Using the Global State
// import { useGlobalState, useGlobalDispatch } from "./GlobalProvider";

// function TodoList() {
//   const { todos, loading } = useGlobalState();
//   const dispatch = useGlobalDispatch();

//   const addTodo = async (title) => {
//     dispatch({ type: "SET_LOADING", payload: true });

//     // Async action example
//     await new Promise((r) => setTimeout(r, 500)); // simulate API

//     dispatch({ type: "ADD_TODO", payload: { id: Date.now(), title } });
//     dispatch({ type: "SET_LOADING", payload: false });
//   };

//   return (
```

```
2133  //     <div>
2134  //       {loading && <p>Loading...</p>}
2135  //       <ul>
2136  //         {todos.map((todo) => (
2137  //           <li key={todo.id}>{todo.title}</li>
2138  //         ))}
2139  //       </ul>
2140  //       <button onClick={() => addTodo("New Todo")}>Add Todo</button>
2141  //     </div>
2142  //   );
2143  // }
2144  // ____________________________________
2145  // 6. Step 5: Async Actions & Middleware
2146  // •    Async actions: Dispatch a function instead of a plain object:
2147  // const fetchUser = (userId) => async (dispatch) => {
2148  //   dispatch({ type: "SET_LOADING", payload: true });
2149  //   try {
2150  //     const res = await fetch(`/api/user/${userId}`);
2151  //     const data = await res.json();
2152  //     dispatch({ type: "SET_USER", payload: data });
2153  //   } catch (err) {
2154  //     dispatch({ type: "SET_ERROR", payload: err });
2155  //   } finally {
2156  //     dispatch({ type: "SET_LOADING", payload: false });
2157  //   }
2158  // };
2159  // •    Logging middleware: Already implemented via console.log in enhanced dispatch.
2160  // •    You can chain additional middleware easily (like analytics, error tracking, etc.
2161  // ____________________________________
2162  // 7. Key Features
2163  // Feature  Implementation
2164  // Global state useReducer inside a Context.Provider
2165  // Dispatch middleware  Enhanced dispatch function to intercept actions
2166  // Async actions    Dispatch function can accept a function (dispatch, getState) => {}
2167  // Logging  Simple console.log middleware inside dispatch
2168  // Scalability  Works for multiple reducers if combined via combineReducers pattern
2169  // ____________________________________
2170  // ✅ Advantages
2171  // •    No external dependencies.
2172  // •    Fully typed if using TypeScript.
2173  // •    Supports async actions, logging, and caching.
2174  // •    Easy to extend with additional middleware (e.g., analytics, performance metrics)
2175
2176  //  Implement a form validation system with real-time feedback.
2177  // // - How would you structure validation rules and error messages?
2178  // // - What about conditional fields and dynamic form schemas?
2179
2180  // 1. Core Concepts
2181  // 1.   State Management
2182  // o    values: current input values.
2183  // o    errors: error messages for each field.
2184  // o    touched: track which fields have been interacted with.
2185  // 2    Validation Rules
```

```
// 2.   Validation Rules
// o    Define a schema where each field has rules (required, min/max length, pattern, c
// o    Rules return error messages or null if valid.
// 3.   Dynamic / Conditional Fields
// o    Form schema can be dynamic (fields added/removed based on other field values).
// ____________________________________________
// 2. Validation Schema Example
// const formSchema = {
//   username: {
//     label: "Username",
//     validators: [
//       { rule: (val) => !!val, message: "Username is required" },
//       { rule: (val) => val.length >= 3, message: "Minimum 3 characters" },
//     ],
//   },
//   email: {
//     label: "Email",
//     validators: [
//       { rule: (val) => !!val, message: "Email is required" },
//       { rule: (val) => /^\S+@\S+\.\S+$/.test(val), message: "Invalid email" },
//     ],
//   },
//   password: {
//     label: "Password",
//     validators: [
//       { rule: (val) => !!val, message: "Password is required" },
//       { rule: (val) => val.length >= 6, message: "Minimum 6 characters" },
//     ],
//   },
//   confirmPassword: {
//     label: "Confirm Password",
//     validators: [
//       { rule: (val, values) => val === values.password, message: "Passwords do not ma
//     ],
//     // Conditional: only show if password has value
//     showIf: (values) => !!values.password,
//   },
// };
// ____________________________________________
// 3. Custom Hook: useForm
// import { useState, useEffect } from "react";

// function useForm(schema, initialValues = {}) {
//   const [values, setValues] = useState(initialValues);
//   const [errors, setErrors] = useState({});
//   const [touched, setTouched] = useState({});

//   const validateField = (name, value) => {
//     const field = schema[name];
//     if (!field) return null;
//     for (let { rule, message } of field.validators) {
//       if (!rule(value, values)) return message;
//     }
```

```
2238  //     return null;
2239  //   };
2240  
2241  //   const handleChange = (e) => {
2242  //     const { name, value } = e.target;
2243  //     setValues((prev) => ({ ...prev, [name]: value }));
2244  //     setTouched((prev) => ({ ...prev, [name]: true }));
2245  
2246  //     // Real-time validation
2247  //     setErrors((prev) => ({ ...prev, [name]: validateField(name, value) }));
2248  //   };
2249  
2250  //   const handleBlur = (e) => {
2251  //     const { name, value } = e.target;
2252  //     setTouched((prev) => ({ ...prev, [name]: true }));
2253  //     setErrors((prev) => ({ ...prev, [name]: validateField(name, value) }));
2254  //   };
2255  
2256  //   const validateForm = () => {
2257  //     const newErrors = {};
2258  //     Object.keys(schema).forEach((key) => {
2259  //       // Only validate fields that are currently visible
2260  //       if (!schema[key].showIf || schema[key].showIf(values)) {
2261  //         const error = validateField(key, values[key]);
2262  //         if (error) newErrors[key] = error;
2263  //       }
2264  //     });
2265  //     setErrors(newErrors);
2266  //     return Object.keys(newErrors).length === 0;
2267  //   };
2268  
2269  //   return {
2270  //     values,
2271  //     errors,
2272  //     touched,
2273  //     handleChange,
2274  //     handleBlur,
2275  //     validateForm,
2276  //     setValues,
2277  //   };
2278  // }
2279  
2280  // export default useForm;
2281  // ____________________________________________
2282  // 4. Form Component Usage
2283  // import useForm from "./useForm";
2284  
2285  // function SignupForm() {
2286  //   const { values, errors, touched, handleChange, handleBlur, validateForm } = useForm
2287  
2288  //   const handleSubmit = (e) => {
2289  //     e.preventDefault();
```

```
2290    //     if (validateForm()) {
2291    //       console.log("Form submitted:", values);
2292    //     }
2293    //   };
2294    
2295    //   return (
2296    //     <form onSubmit={handleSubmit}>
2297    //       {Object.keys(formSchema).map((key) => {
2298    //         const field = formSchema[key];
2299    //         // Handle conditional fields
2300    //         if (field.showIf && !field.showIf(values)) return null;
2301    
2302    //         return (
2303    //           <div key={key} style={{ marginBottom: "1rem" }}>
2304    //             <label>{field.label}</label>
2305    //             <input
2306    //               name={key}
2307    //               value={values[key] || ""}
2308    //               onChange={handleChange}
2309    //               onBlur={handleBlur}
2310    //             />
2311    //             {touched[key] && errors[key] && <p style={{ color: "red" }}>{errors[key]}
2312    //           </div>
2313    //         );
2314    //       })}
2315    //       <button type="submit">Submit</button>
2316    //     </form>
2317    //   );
2318    // }
2319    // ____________________________________
2320    // 5. Key Features
2321    // Feature  Implementation
2322    // Real-time feedback   Validate field on onChange and onBlur.
2323    // Validation rules Array of { rule: fn, message: string } for flexibility.
2324    // Error messages   Stored in errors state and displayed under fields.
2325    // Conditional fields   showIf function in schema controls visibility.
2326    // Dynamic form schema  Add/remove fields in formSchema dynamically based on state or pr
2327    // Full form validation validateForm validates all visible fields before submission.
2328    // ____________________________________
2329    // ✅ Advantages
2330    // •    Fully reusable for multiple forms.
2331    // •    Supports dynamic fields and conditional logic.
2332    // •    Real-time feedback improves UX.
2333    // •    Easy to extend for custom validations, async checks, or integration with APIs.
2334    
2335    // Create a drag and drop interface for reordering lists.
2336    // // - Discuss visual feedback during dragging operations
2337    // // - What about touch device support and accessibility?
2338    
2339    // 1. Core Concepts
2340    // •    State Management: Track the list items and their order.
2341    // •    Drag Events: Use onDragStart, onDragOver, onDrop for desktop; touchstart, touchm
2342    // •    Visual Feedback: Highlight the dragged item and show a placeholder where it will
```

```
// •    Accessibility: Support keyboard navigation and ARIA attributes.
// ____________________________________
// 2. Basic Implementation (Desktop)
// import { useState } from "react";

// function DragDropList({ initialItems }) {
//   const [items, setItems] = useState(initialItems);
//   const [draggedIndex, setDraggedIndex] = useState(null);

//   const handleDragStart = (index) => {
//     setDraggedIndex(index);
//   };

//   const handleDragOver = (e, index) => {
//     e.preventDefault();
//     if (index === draggedIndex) return;
//     const newItems = [...items];
//     const [draggedItem] = newItems.splice(draggedIndex, 1);
//     newItems.splice(index, 0, draggedItem);
//     setDraggedIndex(index);
//     setItems(newItems);
//   };

//   const handleDragEnd = () => {
//     setDraggedIndex(null);
//   };

//   return (
//     <ul style={{ listStyle: "none", padding: 0 }}>
//       {items.map((item, index) => (
//         <li
//           key={item.id}
//           draggable
//           onDragStart={() => handleDragStart(index)}
//           onDragOver={(e) => handleDragOver(e, index)}
//           onDragEnd={handleDragEnd}
//           style={{
//             padding: "8px 16px",
//             marginBottom: "4px",
//             border: "1px solid #ccc",
//             borderRadius: "4px",
//             backgroundColor: index === draggedIndex ? "#e0f7fa" : "#fff",
//             cursor: "move",
//           }}
//         >
//           {item.content}
//         </li>
//       ))}
//     </ul>
//   );
// }
// ____________________________________
```

```
// 3. Visual Feedback
// •    Highlight the dragged item with a different background (#e0f7fa).
// •    Optionally, insert a placeholder to indicate where the item will be dropped.
// •    Use CSS transitions for smooth movement:
// li {
//   transition: transform 0.2s ease;
// }
// ____________________________________________
// 4. Touch Device Support
// •    Desktop drag events don’t work on mobile. Use pointer events or touch events:
// const handleTouchStart = (index) => setDraggedIndex(index);

// const handleTouchMove = (e) => {
//   const touch = e.touches[0];
//   const targetElement = document.elementFromPoint(touch.clientX, touch.clientY);
//   const index = Number(targetElement?.dataset?.index);
//   if (index !== undefined && index !== draggedIndex) {
//     const newItems = [...items];
//     const [draggedItem] = newItems.splice(draggedIndex, 1);
//     newItems.splice(index, 0, draggedItem);
//     setDraggedIndex(index);
//     setItems(newItems);
//   }
// };

// const handleTouchEnd = () => setDraggedIndex(null);
// Attach these handlers to each <li> element.
// ____________________________________________
// 5. Accessibility Considerations
// 1.   Keyboard Reordering
// o    Use tabIndex="0" on list items.
// o    Arrow keys move focus and optionally swap items:
// <li
//   tabIndex={0}
//   onKeyDown={(e) => {
//     if (e.key === "ArrowUp" && index > 0) {
//       const newItems = [...items];
//       [newItems[index], newItems[index - 1]] = [newItems[index - 1], newItems[index]]
//       setItems(newItems);
//     }
//     if (e.key === "ArrowDown" && index < items.length - 1) {
//       const newItems = [...items];
//       [newItems[index], newItems[index + 1]] = [newItems[index + 1], newItems[index]]
//       setItems(newItems);
//     }
//   }}
// >
// 2.   ARIA Attributes
// o    role="list" on <ul> and role="listitem" on <li>.
// o    Add aria-grabbed={index === draggedIndex} to indicate which item is being dragge
// ____________________________________________
// 6. Key Features
// Feature  Implementation
```

```
// Feature  Implementation
// Drag & Drop  draggable + onDragStart, onDragOver, onDrop
// Visual Feedback  Highlight dragged item, placeholder, CSS transitions
// Touch Support    touchstart, touchmove, touchend or pointer events
// Keyboard Accessibility   Arrow key reordering, tabIndex, onKeyDown
// ARIA Compliance  role="list", role="listitem", aria-grabbed
// ____________________________________
// 7. Advanced Enhancements
// •    Smooth animations using react-spring or framer-motion.
// •    Snap-to-grid or sortable grids.
// •    Multi-item selection and drag.
// •    Combine with state management for saving order in backend.

// Design a notification/toast system.
// // - How would you manage multiple notifications and their lifecycle?
// // - How would you implement different notification types and priorities?

// . Core Concepts
// 1.   State Management:
// o    Maintain a list of notifications in global state or a context.
// o    Each notification has an id, message, type, priority, and duration.
// 2.   Notification Lifecycle:
// o    Auto-dismiss after duration.
// o    Allow manual dismissal.
// 3.   Notification Types & Priorities:
// o    Types: success, error, info, warning.
// o    Priorities can determine stack order or visual prominence.
// 4.   Display & Animation:
// o    Stack notifications vertically, optionally with slide/fade animations.
// ____________________________________
// 2. Notification Context & Provider
// import { createContext, useContext, useState, useCallback } from "react";
// import { v4 as uuidv4 } from "uuid";

// const NotificationContext = createContext();

// export const useNotifications = () => useContext(NotificationContext);

// export const NotificationProvider = ({ children }) => {
//   const [notifications, setNotifications] = useState([]);

//   const addNotification = useCallback(({ message, type = "info", duration = 3000, pri
//     const id = uuidv4();
//     const newNotification = { id, message, type, duration, priority };
//     setNotifications((prev) => [...prev, newNotification]);

//     // Auto-remove after duration
//     setTimeout(() => removeNotification(id), duration);
//   }, []);

//   const removeNotification = useCallback((id) => {
//     setNotifications((prev) => prev.filter((n) => n.id !== id));
//   }, []);
```

```
2500
2501    //   return (
2502    //     <NotificationContext.Provider value={{ notifications, addNotification, removeNoti
2503    //       {children}
2504    //     </NotificationContext.Provider>
2505    //   );
2506    // };
2507    // ____________________________________________
2508    // 3. Notification Component
2509    // function Toast({ notification, onClose }) {
2510    //   const { message, type } = notification;
2511
2512    //   const bgColors = {
2513    //     success: "#4caf50",
2514    //     error: "#f44336",
2515    //     info: "#2196f3",
2516    //     warning: "#ff9800",
2517    //   };
2518
2519    //   return (
2520    //     <div
2521    //       style={{
2522    //         background: bgColors[type] || "#333",
2523    //         color: "#fff",
2524    //         padding: "12px 20px",
2525    //         marginBottom: "8px",
2526    //         borderRadius: "4px",
2527    //         boxShadow: "0 2px 6px rgba(0,0,0,0.2)",
2528    //         display: "flex",
2529    //         justifyContent: "space-between",
2530    //         alignItems: "center",
2531    //         cursor: "pointer",
2532    //         minWidth: "250px",
2533    //       }}
2534    //       onClick={onClose}
2535    //     >
2536    //       <span>{message}</span>
2537    //       <button style={{ marginLeft: "12px", background: "transparent", border: "none",
2538    //         &times;
2539    //       </button>
2540    //     </div>
2541    //   );
2542    // }
2543    // ____________________________________________
2544    // 4. Notification Container
2545    // import { useNotifications } from "./NotificationProvider";
2546
2547    // export default function NotificationContainer() {
2548    //   const { notifications, removeNotification } = useNotifications();
2549
2550    //   // Optional: sort by priority (higher first)
2551    //   const sortedNotifications = [...notifications].sort((a, b) => b.priority - a.priori
```

```
2552
2553    //   return (
2554    //     <div style={{ position: "fixed", top: 20, right: 20, zIndex: 1000 }}>
2555    //       {sortedNotifications.map((n) => (
2556    //         <Toast key={n.id} notification={n} onClose={() => removeNotification(n.id)} /
2557    //       ))}
2558    //     </div>
2559    //   );
2560    // }
2561    // ____________________________________________
2562    // 5. Usage Example
2563    // import { NotificationProvider, useNotifications } from "./NotificationProvider";
2564    // import NotificationContainer from "./NotificationContainer";
2565
2566    // function App() {
2567    //   const { addNotification } = useNotifications();
2568
2569    //   return (
2570    //     <div>
2571    //       <button onClick={() => addNotification({ message: "Success!", type: "success" }
2572    //         Show Success
2573    //       </button>
2574    //       <button onClick={() => addNotification({ message: "Error occurred!", type: "err
2575    //         Show High Priority Error
2576    //       </button>
2577    //       <NotificationContainer />
2578    //     </div>
2579    //   );
2580    // }
2581
2582    // export default function Root() {
2583    //   return (
2584    //     <NotificationProvider>
2585    //       <App />
2586    //     </NotificationProvider>
2587    //   );
2588    // }
2589    // ____________________________________________
2590    // 6. Key Features
2591    // Feature  Implementation
2592    // Multiple notifications   Stored in an array in context and rendered in container.
2593    // Lifecycle management Auto-dismiss via setTimeout and manual dismissal.
2594    // Types & Styles   success, error, info, warning mapped to different colors/styles.
2595    // Priorities   Sort notifications by priority before rendering.
2596    // Click to dismiss Each toast has an onClose handler.
2597    // Positioning & stacking   Fixed container with vertical stacking (top-right).
2598    // ____________________________________________
2599    // 7. Advanced Enhancements
2600    // 1.   Animations: Fade/slide with framer-motion or CSS transitions.
2601    // 2.   Queue system: Limit max visible notifications and queue the rest.
2602    // 3.   Custom durations per type: e.g., errors stay longer than info.
2603    // 4.   Accessibility:
2604    // o    role="status" for screen readers.
```

```
// o    Announce new notifications via aria-live="polite".
// ____________________________________________
// This system is scalable, type-aware, priority-aware, and can handle multiple notifica
// I can also build a version with animation and a max-stack queue so older notificatior

// Implement a multi-step form wizard with navigation.
// // - How would you manage form state across multiple steps?
// // - What about saving draft progress and resuming later?

// 1. Core Concepts
// 1.   State Management
// o    Use a single form state object for all steps.
// o    Track the current step index.
// 2.   Navigation
// o    Next / Previous buttons to move between steps.
// o    Optional validation before moving forward.
// 3.   Draft Saving
// o    Store state in localStorage or backend.
// o    Load draft when the user returns.
// 4.   Dynamic Step Rendering
// o    Render only the active step component.
// o    Each step accesses/modifies the shared form state.
// ____________________________________________
// 2. Form State Hook
// import { useState, useEffect } from "react";

// function useMultiStepForm(initialValues = {}, draftKey = "formWizardDraft") {
//   const [currentStep, setCurrentStep] = useState(0);
//   const [formValues, setFormValues] = useState(() => {
//     // Load draft if exists
//     const draft = localStorage.getItem(draftKey);
//     return draft ? JSON.parse(draft) : initialValues;
//   });

//   useEffect(() => {
//     // Save draft on changes
//     localStorage.setItem(draftKey, JSON.stringify(formValues));
//   }, [formValues, draftKey]);

//   const nextStep = () => setCurrentStep((prev) => prev + 1);
//   const prevStep = () => setCurrentStep((prev) => Math.max(prev - 1, 0));

//   const updateField = (name, value) => {
//     setFormValues((prev) => ({ ...prev, [name]: value }));
//   };

//   const resetForm = () => {
//     setFormValues(initialValues);
//     setCurrentStep(0);
//     localStorage.removeItem(draftKey);
//   };

```

```
2657  //   return { currentStep, formValues, updateField, nextStep, prevStep, resetForm, setCu
2658  // }
2659  // ____________________________________________
2660  // 3. Step Components
2661  // Each step receives formValues and updateField:
2662  // function Step1({ formValues, updateField }) {
2663  //   return (
2664  //     <div>
2665  //       <label>
2666  //         First Name:
2667  //         <input
2668  //           type="text"
2669  //           value={formValues.firstName || ""}
2670  //           onChange={(e) => updateField("firstName", e.target.value)}
2671  //         />
2672  //       </label>
2673  //       <label>
2674  //         Last Name:
2675  //         <input
2676  //           type="text"
2677  //           value={formValues.lastName || ""}
2678  //           onChange={(e) => updateField("lastName", e.target.value)}
2679  //         />
2680  //       </label>
2681  //     </div>
2682  //   );
2683  // }
2684
2685  // function Step2({ formValues, updateField }) {
2686  //   return (
2687  //     <div>
2688  //       <label>
2689  //         Email:
2690  //         <input
2691  //           type="email"
2692  //           value={formValues.email || ""}
2693  //           onChange={(e) => updateField("email", e.target.value)}
2694  //         />
2695  //       </label>
2696  //       <label>
2697  //         Phone:
2698  //         <input
2699  //           type="tel"
2700  //           value={formValues.phone || ""}
2701  //           onChange={(e) => updateField("phone", e.target.value)}
2702  //         />
2703  //       </label>
2704  //     </div>
2705  //   );
2706  // }
2707  // ____________________________________________
2708  // 4. Wizard Component
2709  // function FormWizard() {
```

```
// function FormWizard() {
//   const { currentStep, formValues, updateField, nextStep, prevStep, resetForm } = use
//     firstName: "",
//     lastName: "",
//     email: "",
//     phone: "",
//   });

//   const steps = [
//     <Step1 formValues={formValues} updateField={updateField} />,
//     <Step2 formValues={formValues} updateField={updateField} />,
//   ];

//   const handleSubmit = () => {
//     console.log("Form submitted:", formValues);
//     resetForm();
//   };

//   return (
//     <div>
//       <h2>Step {currentStep + 1}</h2>
//       {steps[currentStep]}
//       <div style={{ marginTop: "1rem" }}>
//         {currentStep > 0 && <button onClick={prevStep}>Previous</button>}
//         {currentStep < steps.length - 1 && <button onClick={nextStep}>Next</button>}
//         {currentStep === steps.length - 1 && <button onClick={handleSubmit}>Submit</b
//       </div>
//     </div>
//   );
// }
// ____________________________________________
// 5. Saving Draft Progress
// •    State is saved in localStorage inside the useEffect of useMultiStepForm.
// •    Users can leave and return, and the wizard restores their progress automatically
// •    To reset, resetForm clears both state and localStorage.
// ____________________________________________
// 6. Key Features
// Feature  Implementation
// Multi-step navigation    currentStep with nextStep / prevStep
// Shared form state    formValues object passed to all steps
// Draft saving localStorage sync in useEffect
// Conditional steps    Render steps dynamically using steps array
// Validation (optional)    Validate fields before calling nextStep()
// Submit handling  Submit final data from the last step
// ____________________________________________
// 7. Enhancements
// 1.   Dynamic steps – Skip steps based on previous answers.
// 2.   Progress bar – Show percentage completion.
// 3.   Async validation – Validate data with API before proceeding.
// 4.   Server-side draft save – For multi-device resuming.
// 5.   Keyboard navigation & accessibility – Tab and Enter to move between steps.
// ____________________________________________
// This system provides a fully functional multi-step form, persists drafts, and is easy
```

```
// I can also create a version with progress bar, conditional steps, and async validatic

// How would you optimize a component that renders thousands of items?
// // - How would you implement component-level code splitting?
// // - How would you profile and identify performance bottlenecks?

// 1. Challenges with Thousands of Items
// •    Rendering all items at once causes slow initial render and high memory usage.
// •    Re-rendering every item on state changes can be costly.
// •    Event listeners or heavy child components amplify the problem.
// ____________________________________________
// 2. Optimization Strategies
// a) Virtualization (Windowing)
// Render only the items visible in the viewport plus a buffer.
// •    Libraries: react-window, react-virtualized
// •    Example using react-window:
// import { FixedSizeList as List } from "react-window";

// function Item({ index, style, data }) {
//   const item = data[index];
//   return (
//     <div style={style}>
//       {item.name}
//     </div>
//   );
// }

// export default function VirtualizedList({ items }) {
//   return (
//     <List
//       height={600}           // viewport height
//       itemCount={items.length}
//       itemSize={50}          // fixed item height
//       width="100%"
//       itemData={items}
//     >
//       {Item}
//     </List>
//   );
// }
// ✅ Benefit: Renders only ~20–50 items at a time instead of thousands.
// ____________________________________________
// b) Memoization
// •    Use React.memo for child components that don't need to re-render unless props ch
// const Item = React.memo(({ item }) => {
//   return <div>{item.name}</div>;
// });
// •    Use useCallback and useMemo to prevent unnecessary re-renders of functions and c
// ____________________________________________
// c) Component-Level Code Splitting
// •    Split large or rarely-used components into lazy-loaded chunks:
// import { Suspense, lazy } from "react";
```

```
2814
2815    // const HeavyComponent = lazy(() => import("./HeavyComponent"));
2816
2817    // function App() {
2818    //   return (
2819    //     <Suspense fallback={<div>Loading...</div>}>
2820    //       <HeavyComponent />
2821    //     </Suspense>
2822    //   );
2823    // }
2824    // •    In a large list, you can lazy-load complex item renderers if each item is heavy.
2825    // _____________________________________
2826    // d) Avoid Inline Functions & Objects
2827    // •    Inline functions/objects cause child components to re-render:
2828    // // BAD
2829    // <Item onClick={() => handleClick(item.id)} />
2830
2831    // // BETTER
2832    // const handleClickMemo = useCallback((id) => { ... }, []);
2833    // <Item onClick={handleClickMemo} />
2834    // _____________________________________
2835    // e) Pagination / Infinite Scroll
2836    // •    Instead of loading all items at once, load chunks from API using infinite scroll
2837    // •    Combine with virtualization for maximum efficiency.
2838    // _____________________________________
2839    // 3. Performance Profiling
2840    // 1.   React DevTools Profiler
2841    // o    Open Profiler tab.
2842    // o    Record interactions and check which components take the most render time.
2843    // o    Identify unnecessary re-renders.
2844    // 2.   Chrome Performance Tab
2845    // o    Record page while scrolling a large list.
2846    // o    Inspect painting, layout, and script execution times.
2847    // 3.   React useMemo / useCallback Checks
2848    // o    Identify functions or computations that are recalculated every render unnecessar
2849    // 4.   Network Profiling
2850    // o    Large datasets can cause delays even before render. Consider server-side paginat
2851    // _____________________________________
2852    // 4. Advanced Optimizations
2853    // •    Dynamic Height Virtualization: Use VariableSizeList from react-window for items
2854    // •    Web Workers: Offload heavy computation (like filtering or sorting thousands of i
2855    // •    Immutable Data Structures: Helps with shallow comparison and memoization.
2856    // •    Batching Updates: Minimize state updates inside loops.
2857    // _____________________________________
2858    // ✅ Summary Table
2859    // Optimization When to Use Benefit
2860    // Virtualization   Large lists Render only visible items
2861    // React.memo / useMemo Re-rendering issues Prevent unnecessary renders
2862    // Code Splitting (React.lazy + Suspense)   Heavy components    Reduce initial bundle si
2863    // Infinite Scroll / Pagination Very large datasets Reduce memory usage and network loac
2864    // Profiling    Always  Identify bottlenecks
2865    // _____________________________________
2866    // This combination ensures your component scales efficiently, reduces memory and render
```

```
2867
2868    // Implement a image lazy loading component with placeholder.
2869    // // - How would you detect when images enter the viewport?
2870    // // - Discuss different placeholder strategies (blur, skeleton, etc.)
2871    // // - How would you handle loading errors and retry mechanisms?
2872
2873    // 1. Detecting when images enter the viewport
2874    // The modern, efficient way is to use the Intersection Observer API, which allows you t
2875    // ________________________________________
2876    // 2. Placeholder strategies
2877    // •    Blurred placeholder: Low-quality, blurred version of the image.
2878    // •    Skeleton placeholder: Grey box or shimmer effect.
2879    // •    Solid color or icon: Minimal approach for very fast loading.
2880    // ________________________________________
2881    // 3. Handling errors and retry
2882    // •    Fallback image if the original fails.
2883    // •    Retry mechanism (with limited attempts) using state.
2884    // ________________________________________
2885    // 4. React LazyImage Component
2886    // import React, { useState, useRef, useEffect } from "react";
2887
2888    // // Props: src, alt, placeholder, retryCount, className
2889    // const LazyImage = ({
2890    //   src,
2891    //   alt,
2892    //   placeholder = "https://via.placeholder.com/300x200?text=Loading...",
2893    //   retryCount = 2,
2894    //   className = "",
2895    // }) => {
2896    //   const [isInView, setIsInView] = useState(false);
2897    //   const [imageSrc, setImageSrc] = useState(placeholder);
2898    //   const [errorCount, setErrorCount] = useState(0);
2899
2900    //   const imgRef = useRef();
2901
2902    //   // Intersection Observer
2903    //   useEffect(() => {
2904    //     const observer = new IntersectionObserver(
2905    //       ([entry]) => {
2906    //         if (entry.isIntersecting) {
2907    //           setIsInView(true);
2908    //           observer.disconnect(); // Stop observing once loaded
2909    //         }
2910    //       },
2911    //       { threshold: 0.1 } // Trigger when 10% visible
2912    //     );
2913
2914    //     if (imgRef.current) observer.observe(imgRef.current);
2915
2916    //     return () => {
2917    //       observer.disconnect();
2918    //     };
```

```
2919  //   }, []);
2920
2921  //   // Load actual image
2922  //   useEffect(() => {
2923  //     if (!isInView) return;
2924
2925  //     const img = new Image();
2926  //     img.src = src;
2927
2928  //     img.onload = () => setImageSrc(src);
2929
2930  //     img.onerror = () => {
2931  //       if (errorCount < retryCount) {
2932  //         setErrorCount(prev => prev + 1);
2933  //       } else {
2934  //         setImageSrc("https://via.placeholder.com/300x200?text=Error+Loading");
2935  //       }
2936  //     };
2937  //   }, [isInView, src, errorCount, retryCount]);
2938
2939  //   return (
2940  //     <img
2941  //       ref={imgRef}
2942  //       src={imageSrc}
2943  //       alt={alt}
2944  //       className={`transition-all duration-500 ${imageSrc !== placeholder ? "blur-0" :
2945  //       style={{ width: "100%", height: "auto", objectFit: "cover" }}
2946  //     />
2947  //   );
2948  // };
2949
2950  // export default LazyImage;
2951  // ____________________________________
2952  //  🔹 How it works
2953  // 1.   Viewport detection: Uses IntersectionObserver.
2954  // 2.   Placeholders: Shows placeholder until the image loads.
2955  // 3.   Blur effect: Adds a subtle blur until the real image is loaded.
2956  // 4.   Error handling & retry:
2957  // o    Retries retryCount times on failure.
2958  // o    Falls back to an error image if all attempts fail.
2959  // ____________________________________
2960  //  🔹 Example usage
2961  // <LazyImage
2962  //   src="https://example.com/high-res-image.jpg"
2963  //   alt="Beautiful scenery"
2964  //   placeholder="https://example.com/low-res-image.jpg"
2965  //   retryCount={3}
2966  //   className="rounded-lg shadow-md"
2967  // />
2968  // ____________________________________
2969  //  ✅ Optional improvements
2970  // •    Add a skeleton loader instead of a blurred placeholder.
2971  // •    Support fade-in animation when the real image loads
```

```
// •    Support fade-in animation when the real image loads.
// •    Add progressive loading with multiple quality levels.
// •    Integrate with React Suspense for server-side rendering scenarios.

// Create a data table with sorting, filtering, and pagination.
// // - Discuss client-side vs server-side operations trade-offs.
// // - How would you optimize re-renders when data changes?
// // - What about column configuration and customizable layouts?

// 1. Client-side vs Server-side Operations
// Feature  Client-side Server-side
// Sorting & Filtering  Done in the browser; fast for small datasets    Done via API; ef
// Pagination   Slice data in JS; easy to implement Server provides only requested page
// Pros Simple, instant, no extra API calls Scales to millions of rows
// Cons Memory-heavy for large datasets More complex; requires API support
// Rule of thumb: Use client-side for <5000 rows; server-side for larger datasets.
// ____________________________________________
// 2. Optimizing Re-renders
// •    Use React.memo for rows or cells.
// •    Use useCallback and useMemo to memoize handlers and computed data.
// •    Avoid passing new object references on every render.
// •    Key tables properly with stable row IDs.
// ____________________________________________
// 3. Column Configuration & Customizable Layouts
// •    Define columns as an array of objects { header, accessor, sortable, filterable,
// •    Supports hiding/showing columns dynamically.
// •    Allows custom cell renderers (e.g., badges, buttons, images).
// ____________________________________________
// 4. React DataTable Component Example
// import React, { useState, useMemo } from "react";

// const DataTable = ({ columns, data, pageSize = 5 }) => {
//   const [sortConfig, setSortConfig] = useState({ key: null, direction: "asc" });
//   const [filterText, setFilterText] = useState("");
//   const [currentPage, setCurrentPage] = useState(1);

//   // Filtering
//   const filteredData = useMemo(() => {
//     if (!filterText) return data;
//     return data.filter(row =>
//       columns.some(col => {
//         const value = row[col.accessor];
//         return value?.toString().toLowerCase().includes(filterText.toLowerCase());
//       })
//     );
//   }, [filterText, data, columns]);

//   // Sorting
//   const sortedData = useMemo(() => {
//     if (!sortConfig.key) return filteredData;
//     return [...filteredData].sort((a, b) => {
//       const aValue = a[sortConfig.key];
//       const bValue = b[sortConfig.key];
```

```
3024  //       if (aValue < bValue) return sortConfig.direction === "asc" ? -1 : 1;
3025  //       if (aValue > bValue) return sortConfig.direction === "asc" ? 1 : -1;
3026  //       return 0;
3027  //     });
3028  //   }, [filteredData, sortConfig]);
3029
3030  //   // Pagination
3031  //   const paginatedData = useMemo(() => {
3032  //     const start = (currentPage - 1) * pageSize;
3033  //     return sortedData.slice(start, start + pageSize);
3034  //   }, [sortedData, currentPage, pageSize]);
3035
3036  //   const requestSort = key => {
3037  //     setSortConfig(prev => ({
3038  //       key,
3039  //       direction: prev.key === key && prev.direction === "asc" ? "desc" : "asc",
3040  //     }));
3041  //   };
3042
3043  //   const totalPages = Math.ceil(sortedData.length / pageSize);
3044
3045  //   return (
3046  //     <div>
3047  //       <input
3048  //         type="text"
3049  //         placeholder="Search..."
3050  //         value={filterText}
3051  //         onChange={e => setFilterText(e.target.value)}
3052  //         className="mb-2 p-1 border rounded"
3053  //       />
3054
3055  //       <table className="table-auto border-collapse w-full">
3056  //         <thead>
3057  //           <tr>
3058  //             {columns.map(col => (
3059  //               <th
3060  //                 key={col.accessor}
3061  //                 onClick={() => col.sortable && requestSort(col.accessor)}
3062  //                 className="border px-4 py-2 cursor-pointer"
3063  //               >
3064  //                 {col.header} {sortConfig.key === col.accessor ? (sortConfig.direction
3065  //               </th>
3066  //             ))}
3067  //           </tr>
3068  //         </thead>
3069  //         <tbody>
3070  //           {paginatedData.map((row, idx) => (
3071  //             <tr key={idx} className="hover:bg-gray-100">
3072  //               {columns.map(col => (
3073  //                 <td key={col.accessor} className="border px-4 py-2">
3074  //                   {col.customCell ? col.customCell(row[col.accessor], row) : row[col.
3075  //                 </td>
```

```
3076  //               ))}
3077  //             </tr>
3078  //           ))}
3079  //         </tbody>
3080  //       </table>
3081
3082  //       {/* Pagination Controls */}
3083  //       <div className="mt-2 flex justify-between items-center">
3084  //         <button onClick={() => setCurrentPage(p => Math.max(p - 1, 1))} disabled={cur
3085  //           Previous
3086  //         </button>
3087  //         <span>
3088  //           Page {currentPage} of {totalPages}
3089  //         </span>
3090  //         <button
3091  //           onClick={() => setCurrentPage(p => Math.min(p + 1, totalPages))}
3092  //           disabled={currentPage === totalPages}
3093  //         >
3094  //           Next
3095  //         </button>
3096  //       </div>
3097  //     </div>
3098  //   );
3099  // };
3100
3101  // export default DataTable;
3102  // ____________________________________________
3103  // 5. Example Usage
3104  // const columns = [
3105  //   { header: "ID", accessor: "id", sortable: true },
3106  //   { header: "Name", accessor: "name", sortable: true, filterable: true },
3107  //   { header: "Email", accessor: "email", sortable: true },
3108  //   {
3109  //     header: "Status",
3110  //     accessor: "status",
3111  //     sortable: true,
3112  //     customCell: value => <span className={value === "Active" ? "text-green-500" : "te
3113  //   },
3114  // ];
3115
3116  // const data = [
3117  //   { id: 1, name: "Alice", email: "alice@example.com", status: "Active" },
3118  //   { id: 2, name: "Bob", email: "bob@example.com", status: "Inactive" },
3119  //   { id: 3, name: "Charlie", email: "charlie@example.com", status: "Active" },
3120  //   // ...more rows
3121  // ];
3122
3123  // <DataTable columns={columns} data={data} pageSize={5} />;
3124  // ____________________________________________
3125  // ✅ Optional Enhancements
3126  // 1.   Server-side support: Add API integration for sorting, filtering, and pagination.
3127  // 2.   Column hiding/reordering: Store preferences in local storage.
3128  // 3.   Virtualization: Use react-window or react-virtualized for large datasets to redu
```

```
// 4.   Advanced filtering: Add date ranges, numeric ranges, or multi-select filters.
// ------------------------------------------


// How to cancel the api call when the timeout occurs using abort controller
// Example: Cancel API call on timeout
// // Create an AbortController instance
// const controller = new AbortController();
// const signal = controller.signal;

// // Set a timeout (e.g., 5 seconds)
// const timeout = setTimeout(() => {
//   controller.abort(); // Cancel the API request
// }, 5000);

// fetch('https://api.example.com/data', { signal })
//   .then(response => {
//     if (!response.ok) {
//       throw new Error('Network response was not ok');
//     }
//     return response.json();
//   })
//   .then(data => {
//     console.log('Data received:', data);
//   })
//   .catch(error => {
//     if (error.name === 'AbortError') {
//       console.log('Fetch aborted due to timeout');
//     } else {
//       console.error('Fetch error:', error);
//     }
//   })
//   .finally(() => {
//     clearTimeout(timeout); // Clean up the timeout
//   });
// ____________________________________
// How it works:
// 1.   AbortController creates a signal to communicate abort requests to the fetch API.
// 2.   setTimeout triggers controller.abort() after the given time.
// 3.   The fetch call listens to the signal.
// 4.   If the request is aborted, the promise is rejected with an AbortError.
// 5.   Always clear the timeout in .finally() to prevent memory leaks.
// ____________________________________
// ✅ This approach works with fetch API and other APIs that support AbortSignal.

// Create a component to upload the file in reactjs ?
// Example: File Upload Component in React
// import React, { useState } from 'react';

// const FileUpload = () => {
//   const [file, setFile] = useState(null);
//   const [preview, setPreview] = useState(null);
```

```
3181  //   const [uploading, setUploading] = useState(false);
3182
3183  //   // Handle file selection
3184  //   const handleFileChange = (e) => {
3185  //     const selectedFile = e.target.files[0];
3186  //     setFile(selectedFile);
3187
3188  //     // Show image preview (optional)
3189  //     if (selectedFile && selectedFile.type.startsWith('image/')) {
3190  //       const reader = new FileReader();
3191  //       reader.onloadend = () => {
3192  //         setPreview(reader.result);
3193  //       };
3194  //       reader.readAsDataURL(selectedFile);
3195  //     } else {
3196  //       setPreview(null);
3197  //     }
3198  //   };
3199
3200  //   // Handle file upload
3201  //   const handleUpload = async () => {
3202  //     if (!file) return;
3203
3204  //     setUploading(true);
3205  //     const formData = new FormData();
3206  //     formData.append('file', file);
3207
3208  //     try {
3209  //       const response = await fetch('https://your-api-endpoint.com/upload', {
3210  //         method: 'POST',
3211  //         body: formData,
3212  //       });
3213
3214  //       const data = await response.json();
3215  //       console.log('Upload successful:', data);
3216  //       alert('File uploaded successfully!');
3217  //     } catch (error) {
3218  //       console.error('Upload error:', error);
3219  //       alert('Upload failed!');
3220  //     } finally {
3221  //       setUploading(false);
3222  //       setFile(null);
3223  //       setPreview(null);
3224  //     }
3225  //   };
3226
3227  //   return (
3228  //     <div style={{ maxWidth: '400px', margin: '20px auto' }}>
3229  //       <input type="file" onChange={handleFileChange} />
3230
3231  //       {preview && (
3232  //         <div style={{ marginTop: '10px' }}>
3233  //           <img src={preview} alt="Preview" style={{ width: '100%', maxHeight: '200px'
```

```
//           <img src={preview} alt="Preview" style={{ width: '100%', maxHeight: '200px
//         </div>
//       )}

//       <button
//         onClick={handleUpload}
//         disabled={!file || uploading}
//         style={{ marginTop: '10px', padding: '10px 20px', cursor: 'pointer' }}
//       >
//         {uploading ? 'Uploading...' : 'Upload'}
//       </button>
//     </div>
//   );
// };

// export default FileUpload;
// ____________________________________________
// Features:
// 1.   File selection using <input type="file" />.
// 2.   Preview for images before uploading.
// 3.   Upload progress indicator (simple button text change).
// 4.   Handles API call with fetch and FormData.
// 5.   Resets state after upload.
// ____________________________________________
// If you want, I can also create a more advanced version with drag-and-drop support, mu

// How to create a Custom Hook for Data Fetching with Loading and Error States ?
// Example: useFetch Custom Hook
// import { useState, useEffect } from 'react';

// const useFetch = (url, options = {}) => {
//   const [data, setData] = useState(null);
//   const [loading, setLoading] = useState(true);
//   const [error, setError] = useState(null);

//   useEffect(() => {
//     if (!url) return;

//     const controller = new AbortController();
//     const signal = controller.signal;

//     const fetchData = async () => {
//       setLoading(true);
//       setError(null);
//       try {
//         const response = await fetch(url, { ...options, signal });
//         if (!response.ok) {
//           throw new Error(`Error: ${response.status} ${response.statusText}`);
//         }
//         const result = await response.json();
//         setData(result);
//       } catch (err) {
//         if (err.name !== 'AbortError') {
```

```
3286    //           setError(err.message);
3287    //         }
3288    //       } finally {
3289    //         setLoading(false);
3290    //       }
3291    //     };
3292
3293    //     fetchData();
3294
3295    //     // Cleanup on unmount or URL change
3296    //     return () => controller.abort();
3297    //   }, [url, JSON.stringify(options)]); // Re-run when url or options change
3298
3299    //   return { data, loading, error };
3300    // };
3301
3302    // export default useFetch;
3303    // ____________________________________________
3304    // Usage Example in a Component
3305    // import React from 'react';
3306    // import useFetch from './useFetch';
3307
3308    // const UsersList = () => {
3309    //   const { data, loading, error } = useFetch('https://jsonplaceholder.typicode.com/use
3310
3311    //   if (loading) return <p>Loading...</p>;
3312    //   if (error) return <p>Error: {error}</p>;
3313
3314    //   return (
3315    //     <ul>
3316    //       {data.map(user => (
3317    //         <li key={user.id}>{user.name} ({user.email})</li>
3318    //       ))}
3319    //     </ul>
3320    //   );
3321    // };
3322
3323    // export default UsersList;
3324    // ____________________________________________
3325    // ✅ Features of This Hook:
3326    // 1.   Reusability – Can fetch any URL with optional fetch options.
3327    // 2.   AbortController – Cancels fetch when the component unmounts or URL changes.
3328    // 3.   Loading & Error states – Makes UI handling simple.
3329    // 4.   Clean and minimal – Keeps components focused on UI.
3330
3331    // How will you Manage Environment-Specific Configurations in Reactjs in your project ?
3332    // 1. Using .env Files
3333    // React (Create React App or Vite) supports environment variables via .env files.
3334    // Example:
3335    // •    .env.development
3336    // REACT_APP_API_URL=https://dev.api.example.com
3337    // REACT_APP_ANALYTICS_KEY=dev-key
```

```
// •    .env.production
// REACT_APP_API_URL=https://api.example.com
// REACT_APP_ANALYTICS_KEY=prod-key
// Important: Environment variable names must start with REACT_APP_ in Create React App.
// ____________________________________________
// 2. Accessing Variables in Code
// const apiUrl = process.env.REACT_APP_API_URL;

// fetch(`${apiUrl}/users`)
//   .then(res => res.json())
//   .then(data => console.log(data));
// •    React will automatically replace these variables at build time based on the envi
// ____________________________________________
// 3. Specifying Environment in Scripts
// In package.json, you can define scripts for different environments:
// "scripts": {
//   "start": "react-scripts start", // development
//   "build:prod": "env-cmd -f .env.production react-scripts build",
//   "build:dev": "env-cmd -f .env.development react-scripts build"
// }
// Using packages like env-cmd or cross-env helps manage environments across OSes.
// ____________________________________________
// 4. Dynamic Configuration (Optional)
// Sometimes you need runtime config without rebuilding:
// •    Place a config.json file in public/:
// {
//   "API_URL": "https://dev.api.example.com"
// }
// •    Fetch it at runtime:
// const [config, setConfig] = useState(null);

// useEffect(() => {
//   fetch('/config.json')
//     .then(res => res.json())
//     .then(setConfig);
// }, []);
// This allows changing endpoints without rebuilding the app.
// ____________________________________________
// 5. Best Practices
// •    Never commit secrets (API keys, passwords) directly; use environment variables.
// •    Use separate .env files for each environment.
// •    Validate environment variables to avoid undefined values.
// •    Keep .env files out of src/ to prevent accidental bundling.

// Create a Higher-Order Component (HOC) to Log Props in reactjs ?
// Example: withLogger HOC
// import React from 'react';

// // HOC that logs props
// const withLogger = (WrappedComponent) => {
//   return (props) => {
//     console.log('Props passed to component:', props);
//     return <WrappedComponent {...props} />;
```

```
3391    //   };
3392    // };
3393
3394    // export default withLogger;
3395    // ____________________________________________
3396    // Usage Example
3397    // Suppose you have a simple component:
3398    // const User = ({ name, age }) => {
3399    //   return (
3400    //     <div>
3401    //       <h2>{name}</h2>
3402    //       <p>Age: {age}</p>
3403    //     </div>
3404    //   );
3405    // };
3406    // You can wrap it with the HOC:
3407    // import withLogger from './withLogger';
3408
3409    // const UserWithLogger = withLogger(User);
3410
3411    // export default function App() {
3412    //   return (
3413    //     <div>
3414    //       <UserWithLogger name="Alice" age={25} />
3415    //       <UserWithLogger name="Bob" age={30} />
3416    //     </div>
3417    //   );
3418    // }
3419    // ____________________________________________
3420    // ✅ How it works:
3421    // 1.   withLogger receives a component (WrappedComponent) as input.
3422    // 2.   Returns a new component that:
3423    // o    Logs props to the console.
3424    // o    Passes props to the original component using {...props}.
3425    // 3.   Can be reused to log props for any component.
3426
3427    //  How to update Document Title on Mount in a React Component ?
3428    // Example: Update Document Title on Mount
3429    // import React, { useEffect } from 'react';
3430
3431    // const Page = () => {
3432    //   useEffect(() => {
3433    //     // Runs when component mounts
3434    //     document.title = "Welcome to My Page";
3435
3436    //     // Optional: cleanup on unmount
3437    //     return () => {
3438    //       document.title = "React App"; // Reset title when unmounted
3439    //     };
3440    //   }, []); // Empty dependency array ensures it runs only once on mount
3441
3442    //   return (
```

```
3443  //     <div>
3444  //       <h1>Hello, World!</h1>
3445  //     </div>
3446  //   );
3447  // };
3448
3449  // export default Page;
3450  // ____________________________________
3451  // How it works:
3452  // 1.   useEffect runs after the component mounts.
3453  // 2.   By passing an empty dependency array [], the effect runs only once.
3454  // 3.   You can optionally return a cleanup function to reset the title when the compone
3455  // ____________________________________
3456  // Alternative: Using a Custom Hook
3457  // You can make a reusable hook to update the document title:
3458  // import { useEffect } from 'react';
3459
3460  // const useDocumentTitle = (title) => {
3461  //   useEffect(() => {
3462  //     const originalTitle = document.title;
3463  //     document.title = title;
3464
3465  //     return () => {
3466  //       document.title = originalTitle;
3467  //     };
3468  //   }, [title]);
3469  // };
3470
3471  // export default useDocumentTitle;
3472  // Usage:
3473  // import React from 'react';
3474  // import useDocumentTitle from './useDocumentTitle';
3475
3476  // const Dashboard = () => {
3477  //   useDocumentTitle("Dashboard - My App");
3478
3479  //   return <h1>Dashboard</h1>;
3480  // };
3481  // ✅ This approach is cleaner and reusable across components.
3482
3483  //  How to Implement a Theme Switcher Using Context API ?
3484  // 1. Create a Theme Context
3485  // import React, { createContext, useState, useContext } from 'react';
3486
3487  // // Create the context
3488  // const ThemeContext = createContext();
3489
3490  // // Custom hook for easier access
3491  // export const useTheme = () => useContext(ThemeContext);
3492
3493  // // Provider component
3494  // export const ThemeProvider = ({ children }) => {
3495  //   const [theme, setTheme] = useState('light'); // 'light' or 'dark'
```

```
3495 //   const [theme, setTheme] = useState('light'); // 'light' or 'dark'
3496
3497 //   const toggleTheme = () => {
3498 //     setTheme(prev => (prev === 'light' ? 'dark' : 'light'));
3499 //   };
3500
3501 //   const value = { theme, toggleTheme };
3502
3503 //   return <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>;
3504 // };
3505 // ____________________________________________
3506 // 2. Wrap your App with ThemeProvider
3507 // import React from 'react';
3508 // import ReactDOM from 'react-dom';
3509 // import App from './App';
3510 // import { ThemeProvider } from './ThemeContext';
3511
3512 // ReactDOM.render(
3513 //   <ThemeProvider>
3514 //     <App />
3515 //   </ThemeProvider>,
3516 //   document.getElementById('root')
3517 // );
3518 // ____________________________________________
3519 // 3. Create a Theme Switcher Component
3520 // import React from 'react';
3521 // import { useTheme } from './ThemeContext';
3522
3523 // const ThemeSwitcher = () => {
3524 //   const { theme, toggleTheme } = useTheme();
3525
3526 //   return (
3527 //     <button
3528 //       onClick={toggleTheme}
3529 //       style={{
3530 //         padding: '10px 20px',
3531 //         background: theme === 'light' ? '#333' : '#eee',
3532 //         color: theme === 'light' ? '#fff' : '#000',
3533 //         border: 'none',
3534 //         borderRadius: '5px',
3535 //         cursor: 'pointer',
3536 //         margin: '10px',
3537 //       }}
3538 //     >
3539 //       Switch to {theme === 'light' ? 'Dark' : 'Light'} Mode
3540 //     </button>
3541 //   );
3542 // };
3543
3544 // export default ThemeSwitcher;
3545 // ____________________________________________
3546 // 4. Consume Theme in Components
3547 // import React from 'react';
```

```
// import { useTheme } from './ThemeContext';

// const Content = () => {
//   const { theme } = useTheme();

//   return (
//     <div
//       style={{
//         height: '100vh',
//         display: 'flex',
//         alignItems: 'center',
//         justifyContent: 'center',
//         background: theme === 'light' ? '#fff' : '#222',
//         color: theme === 'light' ? '#000' : '#fff',
//         transition: 'all 0.3s ease',
//       }}
//     >
//       <h1>{theme === 'light' ? 'Light Mode' : 'Dark Mode'}</h1>
//     </div>
//   );
// };

// export default Content;
// ____________________________________
// ✅ Features:
// 1.   Global state using Context API (theme and toggleTheme).
// 2.   Reusable hook useTheme() for easier access.
// 3.   Smooth theme transitions using inline styles or CSS classes.
// 4.   Works across all components without prop drilling.

// How do you access the dom element in reactjs ?
// 1. Using useRef in Functional Components
// import React, { useRef } from 'react';

// const TextInput = () => {
//   const inputRef = useRef(null); // Create a ref

//   const handleFocus = () => {
//     // Access the DOM element and call focus
//     inputRef.current.focus();
//   };

//   return (
//     <div>
//       <input type="text" ref={inputRef} placeholder="Type here..." />
//       <button onClick={handleFocus}>Focus Input</button>
//     </div>
//   );
// };

// export default TextInput;
// Explanation:
```

```
// •    useRef(null) creates a ref object.
// •    ref={inputRef} attaches it to the DOM element.
// •    inputRef.current points to the actual DOM node.
// ____________________________________
// 2. Using createRef in Class Components
// import React, { Component } from 'react';

// class TextInputClass extends Component {
//   constructor(props) {
//     super(props);
//     this.inputRef = React.createRef(); // Create a ref
//   }

//   handleFocus = () => {
//     this.inputRef.current.focus(); // Access the DOM element
//   };

//   render() {
//     return (
//       <div>
//         <input type="text" ref={this.inputRef} placeholder="Type here..." />
//         <button onClick={this.handleFocus}>Focus Input</button>
//       </div>
//     );
//   }
// }

// export default TextInputClass;
// ____________________________________
// 3. Key Points:
// 1.   Refs give direct access to DOM nodes or React elements.
// 2.   Don’t overuse refs; prefer state and props for UI updates.
// 3.   Can be used for:
// o    Focusing inputs
// o    Scrolling
// o    Measuring element dimensions
// o    Integrating third-party libraries that require a DOM node
// ____________________________________
// If you want, I can create an example showing scrolling a list to the bottom using ref

// How to change styles based on condition in react ?
// 1. Using Inline Styles
// You can set the style prop with a JavaScript object and use conditions:
// import React, { useState } from 'react';

// const ConditionalStyle = () => {
//   const [isActive, setIsActive] = useState(false);

//   return (
//     <div>
//       <button onClick={() => setIsActive(!isActive)}>Toggle State</button>

//       <p
```

```
3653    //         style={{
3654    //           color: isActive ? 'white' : 'black',
3655    //           backgroundColor: isActive ? 'green' : 'gray',
3656    //           padding: '10px',
3657    //           borderRadius: '5px',
3658    //         }}
3659    //       >
3660    //         {isActive ? 'Active' : 'Inactive'}
3661    //       </p>
3662    //     </div>
3663    //   );
3664    // };
3665
3666    // export default ConditionalStyle;
3667    // ____________________________________________
3668    // 2. Using CSS Classes
3669    // You can dynamically assign classes using template literals or the classnames library:
3670    // Using Template Literals
3671    // <p className={isActive ? 'active' : 'inactive'}>
3672    //   {isActive ? 'Active' : 'Inactive'}
3673    // </p>
3674    // CSS
3675    // .active {
3676    //   color: white;
3677    //   background-color: green;
3678    // }
3679
3680    // .inactive {
3681    //   color: black;
3682    //   background-color: gray;
3683    // }
3684    // Using classnames (Optional)
3685    // npm install classnames
3686    // import classNames from 'classnames';
3687
3688    // <p className={classNames({ active: isActive, inactive: !isActive })}>
3689    //   {isActive ? 'Active' : 'Inactive'}
3690    // </p>
3691    // ____________________________________________
3692    // 3. Using Styled-Components (CSS-in-JS)
3693    // import styled from 'styled-components';
3694
3695    // const Box = styled.div`
3696    //   padding: 10px;
3697    //   border-radius: 5px;
3698    //   color: ${({ isActive }) => (isActive ? 'white' : 'black')};
3699    //   background-color: ${({ isActive }) => (isActive ? 'green' : 'gray')};
3700    // `;
3701
3702    // const App = () => {
3703    //   const [isActive, setIsActive] = React.useState(false);
3704
```

```
3705  //   return (
3706  //     <div>
3707  //       <button onClick={() => setIsActive(!isActive)}>Toggle</button>
3708  //       <Box isActive={isActive}>{isActive ? 'Active' : 'Inactive'}</Box>
3709  //     </div>
3710  //   );
3711  // };
3712  // ____________________________________
3713  // ✅ Summary
3714  // Approach When to Use
3715  // Inline styles    Quick dynamic styles, small components
3716  // CSS classes  Reusable, cleaner, maintainable styles
3717  // Styled-components    Dynamic styling with component encapsulation
3718
3719  // How to call a method when component is rendered for the first time in react
3720  // 1. Using useEffect in Functional Components
3721  // import React, { useEffect } from 'react';
3722
3723  // const MyComponent = () => {
3724
3725  //   const fetchData = () => {
3726  //     console.log("Component mounted! Fetching data...");
3727  //     // You can call API or other methods here
3728  //   };
3729
3730  //   useEffect(() => {
3731  //     fetchData();
3732  //   }, []); // Empty dependency array ensures it runs only once on mount
3733
3734  //   return <div>Hello, World!</div>;
3735  // };
3736
3737  // export default MyComponent;
3738  // Explanation:
3739  // •    useEffect runs after the component mounts.
3740  // •    Passing an empty dependency array [] ensures it only runs once (on initial rende
3741  // ____________________________________
3742  // 2. Using componentDidMount in Class Components
3743  // import React, { Component } from 'react';
3744
3745  // class MyComponentClass extends Component {
3746  //   componentDidMount() {
3747  //     console.log("Component mounted! Fetching data...");
3748  //     // Call your method here
3749  //   }
3750
3751  //   render() {
3752  //     return <div>Hello, World!</div>;
3753  //   }
3754  // }
3755
3756  // export default MyComponentClass;
3757  // Explanation:
```

```
// Explanation:
// •    componentDidMount runs once after the component is first added to the DOM.
// •    Ideal for API calls, subscriptions, or other initialization logic.
// _____________________________________
// ✅ Notes:
// 1.   Do not call the method directly in render, as it will run on every render.
// 2.   For functional components, always use useEffect with the empty array for first-r
// 3.   For cleanup, you can return a function from useEffect if needed (like unsubscrib

// Give an example of optimization using usecallbacks in react ?
// Absolutely! useCallback is used in React to memoize functions, so they don’t get recr
// Here’s a practical example:
// _____________________________________
// Scenario:
// We have a counter and a child button component that only re-renders when necessary.
// _____________________________________
// 1. Child Component with React.memo
// import React from 'react';

// const Button = React.memo(({ handleClick, children }) => {
//   console.log(`Rendering button: ${children}`);
//   return <button onClick={handleClick}>{children}</button>;
// });

// export default Button;
// •    React.memo prevents re-rendering if props don’t change.
// _____________________________________
// 2. Parent Component Using useCallback
// import React, { useState, useCallback } from 'react';
// import Button from './Button';

// const Counter = () => {
//   const [count, setCount] = useState(0);
//   const [text, setText] = useState('');

//   // Without useCallback, this function would be recreated on every render
//   const increment = useCallback(() => {
//     setCount(prev => prev + 1);
//   }, []); // dependencies: only if needed

//   const decrement = useCallback(() => {
//     setCount(prev => prev - 1);
//   }, []);

//   return (
//     <div>
//       <h1>Count: {count}</h1>
//       <input
//         type="text"
//         placeholder="Type something"
//         value={text}
//         onChange={(e) => setText(e.target.value)}
//       />
```

```
3810    //       <Button handleClick={increment}>Increment</Button>
3811    //       <Button handleClick={decrement}>Decrement</Button>
3812    //     </div>
3813    //   );
3814    // };
3815
3816    // export default Counter;
3817    // ____________________________________________
3818    // ✅ How Optimization Works:
3819    // 1.   Without useCallback, every render recreates functions, causing child components
3820    // 2.   With useCallback, increment and decrement are memoized, so React.memo can prever
3821    // 3.   Useful when you have expensive child components or large lists.
3822    // ____________________________________________
3823    // 🔹 Key Points:
3824    // •    useCallback(fn, deps) returns a memoized version of fn that only changes if deps
3825    // •    Often used with React.memo children or useEffect dependencies.
3826    // •    Helps improve performance in large apps by reducing unnecessary renders.
3827
3828    // How to rerender a component on value change in react ?
3829    // In React, components automatically re-render when their state or props change. So to
3830    // Here's a detailed explanation:
3831    // ____________________________________________
3832    // 1. Using useState in Functional Components
3833    // import React, { useState } from 'react';
3834
3835    // const Counter = () => {
3836    //   const [count, setCount] = useState(0);
3837
3838    //   const increment = () => {
3839    //     setCount(prev => prev + 1); // Updates state → triggers re-render
3840    //   };
3841
3842    //   return (
3843    //     <div>
3844    //       <h1>Count: {count}</h1>
3845    //       <button onClick={increment}>Increment</button>
3846    //     </div>
3847    //   );
3848    // };
3849
3850    // export default Counter;
3851    // Explanation:
3852    // •    useState creates a state variable count.
3853    // •    Calling setCount updates the state.
3854    // •    Updating state automatically rerenders the component with the new value.
3855    // ____________________________________________
3856    // 2. Using this.setState in Class Components
3857    // import React, { Component } from 'react';
3858
3859    // class CounterClass extends Component {
3860    //   state = { count: 0 };
3861
```

```
3862    //   increment = () => {
3863    //     this.setState(prevState => ({ count: prevState.count + 1 }));
3864    //   };
3865
3866    //   render() {
3867    //     return (
3868    //       <div>
3869    //         <h1>Count: {this.state.count}</h1>
3870    //         <button onClick={this.increment}>Increment</button>
3871    //       </div>
3872    //     );
3873    //   }
3874    // }
3875
3876    // export default CounterClass;
3877    // Explanation:
3878    // •    this.setState updates the state and triggers a re-render.
3879    // •    The component displays the updated count.
3880    // ____________________________________________
3881    // 3. Rerender on Prop Change
3882    // If a component receives props from its parent, it will also re-render whenever the pr
3883    // const Child = ({ value }) => {
3884    //   console.log('Child rendered');
3885    //   return <p>Value: {value}</p>;
3886    // };
3887
3888    // const Parent = () => {
3889    //   const [count, setCount] = useState(0);
3890
3891    //   return (
3892    //     <div>
3893    //       <Child value={count} />
3894    //       <button onClick={() => setCount(count + 1)}>Increment</button>
3895    //     </div>
3896    //   );
3897    // };
3898    // •    Child re-renders automatically when value prop changes.
3899    // ____________________________________________
3900    // ✅ Key Points:
3901    // 1.   State changes (useState / this.setState) trigger re-renders.
3902    // 2.   Prop changes from parent also trigger re-renders.
3903    // 3.   Avoid directly mutating state; always use setters (setCount) to trigger re-rende
3904
3905    // How to Force a component to rerender with out using useState in react ?
3906    // Yes! In React, you can force a component to re-render even without changing state. He
3907    // ____________________________________________
3908    // 1. Using useReducer with a dummy state
3909    // import React, { useReducer } from 'react';
3910
3911    // const ForceRender = () => {
3912    //   const [, forceUpdate] = useReducer(x => x + 1, 0);
3913
3914    //   return (
```

```
//     <div>
//       <button onClick={forceUpdate}>Force Rerender</button>
//       <p>Rendered at: {new Date().toLocaleTimeString()}</p>
//     </div>
//   );
// };

// export default ForceRender;
// Explanation:
// •    useReducer updates a dummy state by incrementing a number.
// •    Every time forceUpdate() is called, the component re-renders.
// •    This avoids using useState directly.
// ____________________________________
// 2. Using useRef and useEffect hack (less common)
// import React, { useRef, useEffect } from 'react';

// const ForceRender = () => {
//   const renderCount = useRef(0);

//   const forceRender = () => {
//     renderCount.current += 1;
//     // Trigger re-render by updating a state indirectly (use reducer preferred)
//   };

//   return (
//     <div>
//       <button onClick={forceRender}>Force Rerender</button>
//       <p>Render count: {renderCount.current}</p>
//     </div>
//   );
// };

// export default ForceRender;
// Note: This doesn't automatically trigger a re-render. You'd need a state update somew
// ____________________________________
// 3. Class Component: Using forceUpdate()
// import React, { Component } from 'react';

// class ForceRenderClass extends Component {
//   render() {
//     return (
//       <div>
//         <button onClick={() => this.forceUpdate()}>Force Rerender</button>
//         <p>Rendered at: {new Date().toLocaleTimeString()}</p>
//       </div>
//     );
//   }
// }

// export default ForceRenderClass;
// Explanation:
// •    this.forceUpdate() triggers a re-render without changing state or props.
```

```
// •    Useful in some legacy class components.
// ____________________________________________
// ✅ Best Practice:
// •    In functional components, prefer useReducer to force rerenders.
// •    Avoid overusing forced rerenders; usually state/props updates should drive UI up

// How to call a method immediately after state is updated or after component is rerende
// In React, if you want to call a method immediately after state is updated or after a
// ____________________________________________
// 1. Functional Components: Using useEffect
// useEffect can be used to run code after a state changes or after a render.
// import React, { useState, useEffect } from 'react';

// const Counter = () => {
//   const [count, setCount] = useState(0);

//   const handleClick = () => {
//     setCount(count + 1); // update state
//   };

//   // This effect runs after every render where `count` changes
//   useEffect(() => {
//     console.log("State updated! Current count:", count);
//     // You can call any method here
//   }, [count]); // dependency array ensures it runs when `count` changes

//   return (
//     <div>
//       <h1>Count: {count}</h1>
//       <button onClick={handleClick}>Increment</button>
//     </div>
//   );
// };

// export default Counter;
// ✅ Key Points:
// •    The effect runs after the DOM updates with the new state.
// •    You can include multiple dependencies if you want the effect to run on multiple
// ____________________________________________
// 2. Class Components: Using Callback in setState
// In class components, setState can accept a callback function that runs after the stat
// import React, { Component } from 'react';

// class CounterClass extends Component {
//   state = { count: 0 };

//   increment = () => {
//     this.setState(
//       { count: this.state.count + 1 },
//       () => {
//         console.log("State updated! Current count:", this.state.count);
//         // Call your method here
//       }
```

```
//       }
//     );
//   };

//   render() {
//     return (
//       <div>
//         <h1>Count: {this.state.count}</h1>
//         <button onClick={this.increment}>Increment</button>
//       </div>
//     );
//   }
// }

// export default CounterClass;
// ✅ Key Points:
// •    The callback of setState guarantees that your function runs after state update a
// ____________________________________________
// Summary:
// Component Type   How to run code after state update
// Functional   useEffect(() => { ... }, [state])
// Class    setState(updater, callback)
// ____________________________________________
// I can also show a functional component example that mimics setState callback behavior

// How to Create a popup using portal in reactjs ?
// In React, you can create a popup/modal using React Portals, which allow you to render
// Here’s a clean example:
// ____________________________________________
// 1. Create a Portal Root in HTML
// In your public/index.html, add a div for the portal:
// <body>
//   <div id="root"></div>
//   <div id="portal-root"></div> <!-- Portal target -->
// </body>
// ____________________________________________
// 2. Create the Modal Component Using Portal
// import React from 'react';
// import ReactDOM from 'react-dom';
// import './Modal.css'; // optional for styling

// const Modal = ({ isOpen, onClose, children }) => {
//   if (!isOpen) return null;

//   return ReactDOM.createPortal(
//     <div className="modal-overlay">
//       <div className="modal-content">
//         <button className="modal-close" onClick={onClose}>
//           &times;
//         </button>
//         {children}
//       </div>
//     </div>,
```

```
4072    //     document.getElementById('portal-root') // portal target
4073    //   );
4074    // };
4075
4076    // export default Modal;
4077    // ____________________________________
4078    // 3. Modal CSS (Modal.css)
4079    // .modal-overlay {
4080    //   position: fixed;
4081    //   top: 0;
4082    //   left: 0;
4083    //   width: 100%;
4084    //   height: 100%;
4085    //   background: rgba(0, 0, 0, 0.5);
4086    //   display: flex;
4087    //   justify-content: center;
4088    //   align-items: center;
4089    //   z-index: 1000;
4090    // }
4091
4092    // .modal-content {
4093    //   background: #fff;
4094    //   padding: 20px;
4095    //   border-radius: 8px;
4096    //   position: relative;
4097    //   min-width: 300px;
4098    // }
4099
4100    // .modal-close {
4101    //   position: absolute;
4102    //   top: 10px;
4103    //   right: 10px;
4104    //   border: none;
4105    //   background: transparent;
4106    //   font-size: 20px;
4107    //   cursor: pointer;
4108    // }
4109    // ____________________________________
4110    // 4. Use the Modal in a Component
4111    // import React, { useState } from 'react';
4112    // import Modal from './Modal';
4113
4114    // const App = () => {
4115    //   const [isOpen, setIsOpen] = useState(false);
4116
4117    //   return (
4118    //     <div>
4119    //       <h1>React Portal Modal Example</h1>
4120    //       <button onClick={() => setIsOpen(true)}>Open Modal</button>
4121
4122    //       <Modal isOpen={isOpen} onClose={() => setIsOpen(false)}>
4123    //         <h2>Hello Modal!</h2>
```

```
4124    //         <p>This is a popup using React Portal.</p>
4125    //       </Modal>
4126    //     </div>
4127    //   );
4128    // };
4129
4130    // export default App;
4131    // ____________________________________
4132    // ✅ How it Works
4133    // 1.   ReactDOM.createPortal(child, container) renders child into a DOM node outside th
4134    // 2.   This prevents issues with CSS overflow, z-index, or stacking context.
4135    // 3.   The modal is fully controlled by the parent via props (isOpen, onClose).
4136
4137    // Which lifecycle hooks in class component are replaced with useEffect in functional co
4138    // In React, the useEffect hook in functional components can replace most of the lifecyc
4139    // ____________________________________
4140    // 1. Mounting
4141    // Class Component Lifecycle    Functional Component with useEffect
4142    // componentDidMount()  useEffect(() => { /* code */ }, [])
4143    // •    Run once after the component is mounted.
4144    // •    Empty dependency array [] ensures it only runs once.
4145    // useEffect(() => {
4146    //   console.log('Component mounted!');
4147    // }, []);
4148    // ____________________________________
4149    // 2. Updating
4150    // Class Component Lifecycle    Functional Component with useEffect
4151    // componentDidUpdate(prevProps, prevState) useEffect(() => { /* code */ }, [dep1, dep2]
4152    // •    Runs when specified dependencies change.
4153    // •    useEffect with dependency array triggers on updates.
4154    // useEffect(() => {
4155    //   console.log('Count changed:', count);
4156    // }, [count]);
4157    // ____________________________________
4158    // 3. Unmounting
4159    // Class Component Lifecycle    Functional Component with useEffect
4160    // componentWillUnmount()   useEffect(() => { return () => { /* cleanup */ } }, [])
4161    // •    Return a cleanup function inside useEffect.
4162    // •    Runs when the component unmounts.
4163    // useEffect(() => {
4164    //   const timer = setInterval(() => console.log('tick'), 1000);
4165
4166    //   return () => clearInterval(timer); // cleanup on unmount
4167    // }, []);
4168    // ____________________________________
4169    // 4. Combined Mount + Update + Unmount
4170    // •    useEffect can handle all three phases depending on the dependency array.
4171    // useEffect(() => {
4172    //   console.log('Mount or dependency changed');
4173
4174    //   return () => console.log('Cleanup on dependency change or unmount');
4175    // }, [dependency]);
4176    //
```

```
// ✅ Summary Table
// Lifecycle Method Functional (useEffect)
// componentDidMount    useEffect(() => {}, [])
// componentDidUpdate   useEffect(() => {}, [dependencies])
// componentWillUnmount useEffect(() => { return cleanup }, [])
// componentDidMount + update   useEffect(() => {}, [dependencies])
// ____________________________________________
// Tip:
// •    You can have multiple useEffect hooks in a functional component to separate conc

// How to Share data between components using context api and redux ?
// ou can share data between components in React using Context API or Redux, depending c
// ____________________________________________
// 1. Sharing Data Using Context API
// The Context API is great for lightweight global state without prop drilling.
// Step 1: Create a Context
// import React, { createContext, useState, useContext } from 'react';

// const AppContext = createContext();

// export const useAppContext = () => useContext(AppContext);

// export const AppProvider = ({ children }) => {
//   const [user, setUser] = useState({ name: 'Alice' });

//   return (
//     <AppContext.Provider value={{ user, setUser }}>
//       {children}
//     </AppContext.Provider>
//   );
// };
// Step 2: Wrap Your App
// import React from 'react';
// import ReactDOM from 'react-dom';
// import App from './App';
// import { AppProvider } from './AppContext';

// ReactDOM.render(
//   <AppProvider>
//     <App />
//   </AppProvider>,
//   document.getElementById('root')
// );
// Step 3: Access Context in Components
// import React from 'react';
// import { useAppContext } from './AppContext';

// const UserProfile = () => {
//   const { user, setUser } = useAppContext();

//   return (
//     <div>
```

```
4229    //       <h1>{user.name}</h1>
4230    //       <button onClick={() => setUser({ name: 'Bob' })}>Change Name</button>
4231    //     </div>
4232    //   );
4233    // };
4234    // ✅ Context API is simple, no extra library required, ideal for small to medium apps.
4235    // ____________________________________________
4236    // 2. Sharing Data Using Redux
4237    // Redux is more suitable for large-scale apps with complex state logic.
4238    // Step 1: Install Redux
4239    // npm install @reduxjs/toolkit react-redux
4240    // Step 2: Create a Slice
4241    // import { createSlice } from '@reduxjs/toolkit';
4242
4243    // export const userSlice = createSlice({
4244    //   name: 'user',
4245    //   initialState: { name: 'Alice' },
4246    //   reducers: {
4247    //     setName: (state, action) => {
4248    //       state.name = action.payload;
4249    //     },
4250    //   },
4251    // });
4252
4253    // export const { setName } = userSlice.actions;
4254    // export default userSlice.reducer;
4255    // Step 3: Configure Store
4256    // import { configureStore } from '@reduxjs/toolkit';
4257    // import userReducer from './userSlice';
4258
4259    // export const store = configureStore({
4260    //   reducer: { user: userReducer },
4261    // });
4262    // Step 4: Wrap App with Provider
4263    // import React from 'react';
4264    // import ReactDOM from 'react-dom';
4265    // import { Provider } from 'react-redux';
4266    // import App from './App';
4267    // import { store } from './store';
4268
4269    // ReactDOM.render(
4270    //   <Provider store={store}>
4271    //     <App />
4272    //   </Provider>,
4273    //   document.getElementById('root')
4274    // );
4275    // Step 5: Access Redux State in Components
4276    // import React from 'react';
4277    // import { useSelector, useDispatch } from 'react-redux';
4278    // import { setName } from './userSlice';
4279
4280    // const UserProfile = () => {
4281    //   const user = useSelector((state) => state.user);
```

```
//   const user = useSelector((state) => state.user);
//   const dispatch = useDispatch();

//   return (
//     <div>
//       <h1>{user.name}</h1>
//       <button onClick={() => dispatch(setName('Bob'))}>Change Name</button>
//     </div>
//   );
// };
// ✅ Redux provides a predictable state container, devtools, and is better for complex
// ____________________________________________
// Comparison
// Feature  Context API Redux
// Setup Complexity Simple  More setup
// Best For Small/medium apps   Large apps with complex state
// Performance  Can re-render all consumers Optimized with selectors
// DevTools Basic logging   Redux DevTools
// Middleware support   Limited Full support (thunks, saga)
// ____________________________________________
// If you want, I can make a small example app showing both Context API and Redux side b

// Give an examaple of optimization using useMemo and usecallback ?
// Sure! Let’s create a practical React example showing how to optimize performance usir
// ____________________________________________
// Scenario:
// We have a counter and a list of numbers. We want to:
// 1.   Avoid recalculating expensive operations on every render (useMemo).
// 2.   Prevent child components from re-rendering unnecessarily (useCallback + React.me
// ____________________________________________
// 1. Child Component
// import React from 'react';

// // React.memo prevents unnecessary re-renders if props don't change
// const List = React.memo(({ numbers, calculate }) => {
//   console.log('List component rendered');
//   return (
//     <ul>
//       {numbers.map((num, index) => (
//         <li key={index}>
//           {num} → Square: {calculate(num)}
//         </li>
//       ))}
//     </ul>
//   );
// });

// export default List;
// ____________________________________________
// 2. Parent Component Using useMemo and useCallback
// import React, { useState, useMemo, useCallback } from 'react';
// import List from './List';

```

```
// const App = () => {
//   const [count, setCount] = useState(0);
//   const [numbers] = useState([1, 2, 3, 4, 5]);

//   // useCallback memoizes the function so it doesn't recreate on every render
//   const calculateSquare = useCallback((num) => {
//     console.log('Calculating square for', num);
//     return num * num;
//   }, []);

//   // useMemo to memoize expensive calculation (optional here)
//   const doubledCount = useMemo(() => {
//     console.log('Doubling count');
//     return count * 2;
//   }, [count]);

//   return (
//     <div>
//       <h1>Count: {count}</h1>
//       <h2>Doubled Count (memoized): {doubledCount}</h2>
//       <button onClick={() => setCount(count + 1)}>Increment</button>

//       <List numbers={numbers} calculate={calculateSquare} />
//     </div>
//   );
// };

// export default App;
// ____________________________________________
// ✅ How This Optimizes Performance:
// 1.   useCallback
// o    Prevents the calculateSquare function from being recreated on every render.
// o    Ensures React.memo in List works properly, avoiding unnecessary re-renders.
// 2.   useMemo
// o    Memoizes expensive calculations (like doubledCount), only recalculating when dep
// o    Prevents recomputation on every render.
// 3.   React.memo
// o    Ensures the List component only re-renders if its props (numbers or calculate) a
// ____________________________________________
// 🔹 Key Points
// Hook Purpose
// useMemo  Memoize a value to avoid expensive recalculation
// useCallback  Memoize a function to avoid recreating it each render

// Create a counter component using useReducer ?
// 1. Counter Component Using useReducer
// import React, { useReducer } from 'react';

// // Define initial state
// const initialState = { count: 0 };

// // Define reducer function
```

```
// const reducer = (state, action) => {
//   switch (action.type) {
//     case 'INCREMENT':
//       return { count: state.count + 1 };
//     case 'DECREMENT':
//       return { count: state.count - 1 };
//     case 'RESET':
//       return { count: 0 };
//     default:
//       return state;
//   }
// };

// const Counter = () => {
//   const [state, dispatch] = useReducer(reducer, initialState);

//   return (
//     <div style={{ textAlign: 'center', marginTop: '50px' }}>
//       <h1>Count: {state.count}</h1>
//       <button onClick={() => dispatch({ type: 'INCREMENT' })}>Increment</button>
//       <button onClick={() => dispatch({ type: 'DECREMENT' })}>Decrement</button>
//       <button onClick={() => dispatch({ type: 'RESET' })}>Reset</button>
//     </div>
//   );
// };

// export default Counter;
// ____________________________________
// ✅ How It Works:
// 1.   initialState – defines the starting state.
// 2.   reducer – takes the current state and an action, returns the new state.
// 3.   dispatch – triggers state changes by sending an action to the reducer.
// 4.   All state updates are handled in a single place, making it easier to manage comp
// ____________________________________
// This pattern is especially useful when:
// •    You have multiple state variables that update together.
// •    State changes depend on previous state.
// •    You want a more predictable state flow, similar to Redux.

// Create a search textbox filter in reactjs ?
// 1. Search Filter Component
// import React, { useState } from 'react';

// const SearchFilter = () => {
//   const [searchTerm, setSearchTerm] = useState('');

//   const items = [
//     'Apple',
//     'Banana',
//     'Orange',
//     'Mango',
//     'Pineapple',
//     'Grapes',
```

```
4439    //   ];
4440
4441      // Filter items based on search term
4442    //   const filteredItems = items.filter(item =>
4443    //     item.toLowerCase().includes(searchTerm.toLowerCase())
4444    //   );
4445
4446    //   return (
4447    //     <div style={{ padding: '20px', maxWidth: '400px', margin: 'auto' }}>
4448    //       <h2>Search Fruits</h2>
4449    //       <input
4450    //         type="text"
4451    //         placeholder="Search..."
4452    //         value={searchTerm}
4453    //         onChange={e => setSearchTerm(e.target.value)}
4454    //         style={{
4455    //           width: '100%',
4456    //           padding: '10px',
4457    //           marginBottom: '20px',
4458    //           borderRadius: '5px',
4459    //           border: '1px solid #ccc',
4460    //         }}
4461    //       />
4462    //       <ul>
4463    //         {filteredItems.length > 0 ? (
4464    //           filteredItems.map((item, index) => <li key={index}>{item}</li>)
4465    //         ) : (
4466    //           <li>No results found</li>
4467    //         )}
4468    //       </ul>
4469    //     </div>
4470    //   );
4471    // };
4472
4473    // export default SearchFilter;
4474    // ____________________________________
4475    // ✅ How It Works:
4476    // 1.   State: searchTerm stores the current value of the input.
4477    // 2.   Filtering: items.filter(...) checks if each item includes the search term (case-
4478    // 3.   Rendering: The filtered list updates in real-time as the user types.
4479    // 4.   Fallback: Shows “No results found” if nothing matches.
4480    // ____________________________________
4481    // 🔹 Optional Improvements:
4482    // •    Debouncing with useEffect or lodash to reduce filter frequency on large datasets
4483    // •    Highlighting matched text in the list.
4484    // •    Fetching data from API instead of static array.
4485
4486    // How will you implement debounce in reactjs for filtering ?
4487    // 1. Using useEffect with setTimeout
4488    // import React, { useState, useEffect } from 'react';
4489
4490    // const SearchFilterDebounce = () => {
```

```
4491  //   const [searchTerm, setSearchTerm] = useState('');
4492  //   const [debouncedTerm, setDebouncedTerm] = useState('');
4493  //   const [filteredItems, setFilteredItems] = useState([]);
4494  
4495  //   const items = ['Apple', 'Banana', 'Orange', 'Mango', 'Pineapple', 'Grapes'];
4496  
4497  //   // Update debouncedTerm after a delay
4498  //   useEffect(() => {
4499  //     const handler = setTimeout(() => {
4500  //       setDebouncedTerm(searchTerm);
4501  //     }, 500); // 500ms debounce delay
4502  
4503  //     return () => {
4504  //       clearTimeout(handler); // Cleanup on next effect run
4505  //     };
4506  //   }, [searchTerm]);
4507  
4508  //   // Filter items whenever debouncedTerm changes
4509  //   useEffect(() => {
4510  //     setFilteredItems(
4511  //       items.filter(item =>
4512  //         item.toLowerCase().includes(debouncedTerm.toLowerCase())
4513  //       )
4514  //     );
4515  //   }, [debouncedTerm]);
4516  
4517  //   return (
4518  //     <div style={{ padding: '20px', maxWidth: '400px', margin: 'auto' }}>
4519  //       <h2>Debounced Search</h2>
4520  //       <input
4521  //         type="text"
4522  //         placeholder="Search..."
4523  //         value={searchTerm}
4524  //         onChange={e => setSearchTerm(e.target.value)}
4525  //         style={{ width: '100%', padding: '10px', marginBottom: '20px' }}
4526  //       />
4527  //       <ul>
4528  //         {filteredItems.length > 0
4529  //           ? filteredItems.map((item, index) => <li key={index}>{item}</li>)
4530  //           : <li>No results found</li>}
4531  //       </ul>
4532  //     </div>
4533  //   );
4534  // };
4535  
4536  // export default SearchFilterDebounce;
4537  // ____________________________________
4538  // How it Works
4539  // 1.   searchTerm updates on every keystroke.
4540  // 2.   useEffect with setTimeout delays updating debouncedTerm until the user stops typ
4541  // 3.   Cleanup clearTimeout ensures no stale timers remain.
4542  // 4.   Filtering happens only after the debounce delay, reducing unnecessary renders.
4543  //
```

```
// ________________________________________
// 2. Optional: Using lodash.debounce
// npm install lodash
// import { debounce } from 'lodash';
// import React, { useState, useMemo } from 'react';

// const SearchFilterLodash = () => {
//   const [searchTerm, setSearchTerm] = useState('');
//   const items = ['Apple', 'Banana', 'Orange', 'Mango'];

//   const handleChange = (e) => {
//     setSearchTerm(e.target.value);
//   };

  // Memoized debounced function
//   const debouncedChange = useMemo(() => debounce(handleChange, 500), []);

//   return (
//     <input
//       type="text"
//       placeholder="Search..."
//       onChange={debouncedChange}
//     />
//   );
// };
// •    lodash.debounce simplifies debouncing.
// •    useMemo ensures the debounced function is stable across renders.
// ________________________________________
// ✅ Key Points
// •    Debouncing improves performance on large datasets or API calls.
// •    Always clean up timers when using setTimeout.
// •    Can be used for filtering, autocomplete, or API search.


// ------------------------------------------------------------------------------------
















// ----------------------------------------------------------------------------

// In Reactjs L1 & L2 some of these 40+ scenario based questions can be asked by intervi

```

```
4596  // 1. How to cancel the api call when the timeout occurs using abort controller
4597  // 2. Create a component to upload the file in reactjs ?
4598  // 3. How to create a Custom Hook for Data Fetching with Loading and Error States ?
4599  // 4. How will you Manage Environment-Specific Configurations in Reactjs in your project
4600  // 5. Create a Higher-Order Component (HOC) to Log Props in reactjs ?
4601  // 6. How to update Document Title on Mount in a React Component ?
4602  // 7. How to Implement a Theme Switcher Using Context API ?
4603  // 8. How do you access the dom element in reactjs ?
4604  // 9. How to change styles based on condition in react ?
4605  // 10. How to call a method when component is rendered for the first time in react
4606  // 11. Give an example of optimization using usecallbacks in react ?
4607  // 12. How to rerender a component on value change in react ?
4608  // 13. How to Force a component to rerender with out using useState in react ?
4609  // 14. How to call a method immediately after state is updated or after component is rer
4610  // 15. How to Create a popup using portal in reactjs ?
4611  // 16. Which lifecycle hooks in class component are replaced with useEffect in functiona
4612  // 17. How to Share data between components using context api and redux ?
4613  // 18. Give an examaple of optimization using useMemo and usecallback ?
4614  // 19. Create a counter component using useReducer ?
4615  // 20. Create a search textbox filter in reactjs ?
4616  // 21. How will you implement debounce in reactjs for filtering ?
4617  //
4618  // 1. Cancel an API call using AbortController with a timeout
4619  const fetchData = async () => {
4620    const controller = new AbortController();
4621    const timeoutId = setTimeout(() => controller.abort(), 5000); // 5 seconds timeout
4622
4623    try {
4624      const response = await fetch('https://api.example.com/data', {
4625        signal: controller.signal
4626      });
4627      const data = await response.json();
4628      console.log(data);
4629    } catch (error) {
4630      if (error.name === 'AbortError') {
4631        console.log('Request cancelled due to timeout');
4632      } else {
4633        console.error('Fetch error:', error);
4634      }
4635    } finally {
4636      clearTimeout(timeoutId);
4637    }
4638  };
4639
4640  fetchData();
4641
4642  // 2. File Upload Component in React
4643  import React, { useState } from 'react';
4644
4645  function FileUpload() {
4646    const [file, setFile] = useState(null);
4647
```

```
  const handleChange = (e) => {
    setFile(e.target.files[0]);
  };

  const handleUpload = () => {
    const formData = new FormData();
    formData.append('file', file);

    fetch('/upload', {
      method: 'POST',
      body: formData
    })
      .then(res => res.json())
      .then(data => console.log(data))
      .catch(err => console.error(err));
  };

  return (
    <div>
      <input type="file" onChange={handleChange} />
      <button onClick={handleUpload} disabled={!file}>Upload</button>
    </div>
  );
}

export default FileUpload;

// 3. Custom Hook for Data Fetching
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    const fetchData = async () => {
      try {
        const response = await fetch(url, { signal: controller.signal });
        if (!response.ok) throw new Error('Network response was not ok');
        const result = await response.json();
        setData(result);
      } catch (err) {
        if (err.name !== 'AbortError') setError(err);
      } finally {
        setLoading(false);
      }
    };
    fetchData();

    return () => controller.abort();
  }, [url]);
```

```
4701
4702      return { data, loading, error };
4703    }
4704
4705    export default useFetch;
4706
4707    // 4. Environment-Specific Configurations
4708
4709    Use .env files in React:
4710
4711    # .env.development
4712    REACT_APP_API_URL=http://localhost:5000
4713
4714    # .env.production
4715    REACT_APP_API_URL=https://api.example.com
4716
4717
4718    Access in code:
4719
4720    const apiUrl = process.env.REACT_APP_API_URL;
4721
4722
4723    React automatically picks .env.development or .env.production based on npm start or npm
4724
4725    // 5. Higher-Order Component (HOC) to Log Props
4726    import React from 'react';
4727
4728    function withLogger(WrappedComponent) {
4729      return function(props) {
4730        console.log('Props:', props);
4731        return <WrappedComponent {...props} />;
4732      };
4733    }
4734
4735    export default withLogger;
4736
4737    // Usage
4738    // const LoggedButton = withLogger(Button);
4739
4740    // 6. Update Document Title on Mount
4741    import { useEffect } from 'react';
4742
4743    function Page() {
4744      useEffect(() => {
4745        document.title = 'New Page Title';
4746      }, []); // Empty dependency array ensures it runs once
4747
4748      return <h1>My Page</h1>;
4749    }
4750
4751    export default Page;
4752
```

```
// 7. Theme Switcher using Context API
import React, { createContext, useState, useContext } from 'react';

const ThemeContext = createContext();

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => setTheme(prev => (prev === 'light' ? 'dark' : 'light'));

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  return useContext(ThemeContext);
}

// Usage in component
/*
const { theme, toggleTheme } = useTheme();
<div className={theme}>Hello</div>
<button onClick={toggleTheme}>Switch Theme</button>
*/

// 8. Access DOM Element in React
import React, { useRef, useEffect } from 'react';

function FocusInput() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus(); // Access DOM element
  }, []);

  return <input ref={inputRef} placeholder="Auto Focus" />;
}

// 9. Change Styles Conditionally
function Button({ primary }) {
  const style = {
    backgroundColor: primary ? 'blue' : 'gray',
    color: 'white',
    padding: '10px'
  };
  return <button style={style}>Click Me</button>;
}

// 10. Call a Method on First Render
import { useEffect } from 'react';
```

```
function MyComponent() {
  useEffect(() => {
    console.log('Component rendered for the first time');
    fetchData(); // call your method
  }, []); // empty dependency array ensures first render only

  return <div>Hello</div>;
}


Here’s a detailed explanation with practical ReactJS examples for each of your questions

// 11. Example of optimization using useCallback

useCallback prevents unnecessary re-creations of functions between renders:

import React, { useState, useCallback } from 'react';

function Child({ handleClick }) {
  console.log('Child rendered');
  return <button onClick={handleClick}>Click Me</button>;
}

function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {
    console.log('Button clicked');
  }, []); // function reference stays same across renders

  return (
    <div>
      <p>Count: {count}</p>
      <Child handleClick={handleClick} />
      <button onClick={() => setCount(c => c + 1)}>Increment</button>
    </div>
  );
}


Without useCallback, Child would re-render unnecessarily whenever Parent re-renders.

// 12. Rerender a component on value change

React automatically rerenders when state or props change:

import React, { useState } from 'react';

function MyComponent() {
  const [value, setValue] = useState(0);

  return (
```

```
      <div>
        <p>Value: {value}</p>
        <button onClick={() => setValue(value + 1)}>Increment</button>
      </div>
    );
  }


  Changing value triggers rerender.

  // 13. Force rerender without using useState
  import React, { useReducer } from 'react';

  function MyComponent() {
    const [, forceUpdate] = useReducer(x => x + 1, 0);

    return (
      <div>
        <p>Random: {Math.random()}</p>
        <button onClick={forceUpdate}>Force Rerender</button>
      </div>
    );
  }

  // 14. Call method immediately after state update

  Use useEffect that depends on the state:

  import React, { useState, useEffect } from 'react';

  function MyComponent() {
    const [count, setCount] = useState(0);

    useEffect(() => {
      console.log('State updated:', count);
      doSomething();
    }, [count]);

    const doSomething = () => console.log('After state update');

    return <button onClick={() => setCount(c => c + 1)}>Increment</button>;
  }

  // 15. Create a popup using React Portal
  import React from 'react';
  import ReactDOM from 'react-dom';

  function Popup({ children, onClose }) {
    return ReactDOM.createPortal(
      <div style={{
        position: 'fixed', top: 0, left: 0,
        width: '100%', height: '100%',
```

```
4910          background: 'rgba(0,0,0,0.5)',
4911          display: 'flex', justifyContent: 'center', alignItems: 'center'
4912        }}>
4913          <div style={{ background: 'white', padding: 20 }}>
4914            {children}
4915            <button onClick={onClose}>Close</button>
4916          </div>
4917        </div>,
4918        document.getElementById('portal-root') // create <div id="portal-root"></div> in inc
4919      );
4920    }
4921
4922    // 16. Lifecycle hooks replaced by useEffect
4923    Class Component Lifecycle   Functional Component Replacement
4924    componentDidMount   useEffect(() => {}, [])
4925    componentDidUpdate  useEffect(() => {}, [deps])
4926    componentWillUnmount    useEffect(() => return cleanup, [])
4927    // 17. Share data using Context API and Redux
4928
4929    Context API
4930
4931    const MyContext = React.createContext();
4932    function Parent() {
4933      const value = { name: 'Saikrishna' };
4934      return <MyContext.Provider value={value}><Child /></MyContext.Provider>;
4935    }
4936    function Child() {
4937      const ctx = React.useContext(MyContext);
4938      return <p>{ctx.name}</p>;
4939    }
4940
4941
4942    Redux
4943
4944    import { useSelector, useDispatch } from 'react-redux';
4945    const name = useSelector(state => state.name);
4946    const dispatch = useDispatch();
4947    dispatch({ type: 'UPDATE_NAME', payload: 'Saikrishna' });
4948
4949    // 18. Optimization using useMemo and useCallback
4950    import React, { useState, useMemo, useCallback } from 'react';
4951
4952    function ExpensiveCalculation({ num, onClick }) {
4953      const result = useMemo(() => {
4954        console.log('Calculating...');
4955        return num * 2;
4956      }, [num]);
4957
4958      return <button onClick={onClick}>Result: {result}</button>;
4959    }
4960
4961    export default function App() {
4962      const [count, setCount] = useState(0);
```

```
4963
4964      const handleClick = useCallback(() => console.log('Clicked'), []);
4965
4966      return <ExpensiveCalculation num={count} onClick={handleClick} />;
4967    }
4968
4969    // 19. Counter using useReducer
4970    import React, { useReducer } from 'react';
4971
4972    const initialState = { count: 0 };
4973    function reducer(state, action) {
4974      switch(action.type) {
4975        case 'increment': return { count: state.count + 1 };
4976        case 'decrement': return { count: state.count - 1 };
4977        default: return state;
4978      }
4979    }
4980
4981    function Counter() {
4982      const [state, dispatch] = useReducer(reducer, initialState);
4983
4984      return (
4985        <div>
4986          <p>Count: {state.count}</p>
4987          <button onClick={() => dispatch({ type: 'increment' })}>+</button>
4988          <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
4989        </div>
4990      );
4991    }
4992
4993    // 20. Search Textbox Filter
4994    import React, { useState } from 'react';
4995
4996    function SearchFilter() {
4997      const [query, setQuery] = useState('');
4998      const items = ['Apple', 'Banana', 'Orange', 'Mango'];
4999
5000      const filtered = items.filter(item => item.toLowerCase().includes(query.toLowerCase())
5001
5002      return (
5003        <div>
5004          <input value={query} onChange={e => setQuery(e.target.value)} placeholder="Search.
5005          <ul>
5006            {filtered.map(item => <li key={item}>{item}</li>)}
5007          </ul>
5008        </div>
5009      );
5010    }
5011
5012    // 21. Debounce implementation for filtering
5013    import React, { useState, useEffect } from 'react';
5014
```

```
5015  function useDebounce(value, delay) {
5016    const [debouncedValue, setDebouncedValue] = useState(value);
5017
5018    useEffect(() => {
5019      const handler = setTimeout(() => setDebouncedValue(value), delay);
5020      return () => clearTimeout(handler);
5021    }, [value, delay]);
5022
5023    return debouncedValue;
5024  }
5025
5026  function Search() {
5027    const [query, setQuery] = useState('');
5028    const debouncedQuery = useDebounce(query, 500); // 500ms delay
5029    const items = ['Apple', 'Banana', 'Orange', 'Mango'];
5030
5031    const filtered = items.filter(item => item.toLowerCase().includes(debouncedQuery.toLow
5032
5033    return (
5034      <div>
5035        <input value={query} onChange={e => setQuery(e.target.value)} placeholder="Search.
5036        <ul>{filtered.map(item => <li key={item}>{item}</li>)}</ul>
5037      </div>
5038    );
5039  }
5040
5041  // ---------------------------------------------------------------------------
5042
5043  // ⚛️ 53 Most Frequently Asked React Interview Questions (L1–L3)
5044
5045  // 🟢 1. React Fundamentals
5046  // 1️⃣ What is React?
5047  // 2️⃣ What are the key features of React?
5048  // 3️⃣ What is JSX?
5049  // 4️⃣ What is the DOM?
5050  // 5️⃣ What is Virtual DOM?
5051  // 6️⃣ What are props in React?
5052  // 7️⃣ What is state in ReactJS?
5053  // 8️⃣ What is the difference between props and state?
5054  // 9️⃣ What is prop drilling?
5055  // 🔟 How to avoid prop drilling?
5056
5057  // 🔵 2. Component System & Lifecycle
5058
5059  // 1️⃣1️⃣ What is the component lifecycle in React class components?
5060  // 1️⃣2️⃣ What are Pure components in React?
5061  // 1️⃣3️⃣ What are fragments in React?
5062  // 1️⃣4️⃣ What are Refs in React?
5063  // 1️⃣5️⃣ What is forwardRef?
5064  // 1️⃣6️⃣ What are Error Boundaries?
5065  // 1️⃣7️⃣ What are Higher Order Components (HOCs)?
5066  // 1️⃣8️⃣ What are Portals in React?
5067  // 1️⃣9️⃣ What is Strict Mode in React?
```

```
5068
5069    // 🟠 3. Hooks Deep Dive
5070
5071    // 2️⃣0️⃣ What is useState and when to use it?
5072    //  2️⃣1️⃣ What is useReducer and when to use it over useState?
5073    //  2️⃣2️⃣ What is useEffect and which class lifecycle methods does it replace?
5074    //  2️⃣3️⃣ What is useMemo?
5075    //  2️⃣4️⃣ What is useCallback?
5076    //  2️⃣5️⃣ Difference between useMemo and useCallback?
5077    //  2️⃣6️⃣ What are custom hooks?
5078    //  2️⃣7️⃣ What are the different hooks you've used?
5079
5080    // 🧠 4. Practical Coding & Usage
5081
5082    // 2️⃣8️⃣ Create a custom hook for increment/decrement counter
5083    //  2️⃣9️⃣ Practical: Use callback function to send data from child to parent
5084    //  3️⃣0️⃣ Practical: Send data from child to parent using useRef
5085    //  3️⃣1️⃣ Practical: Use Context API with a working example
5086
5087    // 🟣 5. Data Flow, Events & Rendering
5088    // 3️⃣2️⃣ What are synthetic events in React?
5089    //  3️⃣3️⃣ What are controlled vs uncontrolled components?
5090    //  3️⃣4️⃣ What are keys in React?
5091    //  3️⃣5️⃣ What is lazy loading in React?
5092    //  3️⃣6️⃣ What is Suspense in React?
5093    //  3️⃣7️⃣ What is the purpose of the callback function inside setState()?
5094    //  3️⃣8️⃣ Different ways to pass data from child to parent component
5095
5096    // 🟡 6. Context, Redux, and State Management
5097
5098    // 3️⃣9️⃣ What is Context API in React?
5099    //  4️⃣0️⃣ Practical: Example usage of Context API
5100    //  4️⃣1️⃣ Context API vs Redux – when to use what?
5101    //  4️⃣2️⃣ What are prop types? How to apply validation on props?
5102
5103    // 🔴 7. Performance & Patterns
5104
5105    // 4️⃣3️⃣ How do you optimize a React application?
5106    //  4️⃣4️⃣ How do you consume a RESTful JSON API in ReactJS?
5107    //  4️⃣5️⃣ What are different design patterns used in React?
5108    //  4️⃣6️⃣ What are React Mixins?
5109    //  4️⃣7️⃣ What are render props in React?
5110    //  4️⃣8️⃣ What is the difference between React.createElement and React.cloneElement?
5111    //  4️⃣9️⃣ What are the different types of exports and imports in React?
5112    //  5️⃣0️⃣ What are protected routes in React?
5113    //  5️⃣1️⃣ Does React Router support context menus?
5114
5115    // ------------------------------------------------------------------------
5116
5117    // In a React Frontend L2 round the following question was asked from interviewer.
5118
5119    // 🔹 What security features should be taken while designing API's?
```

```
// 🔹 What is API throttling?
// 🔹 How to improve the performance in React Application?
// 🔹 What is debouncing in React?
// 🔹 What if in a react app we need to develop a feature of auto save the inputs of a
// 🔹 Which react library is used to represent JSON data into charts, graphs, etc for b
// 🔹 If you have to inform the backend developers about some API's are failing how wil
// 🔹 Which tool is used to improve code standards in react application to show warning
// 🔹 What are the unit testing tools used in React Application?
// 🔹 How do you handle API test case scenarios in React Application?
// 🔹 Give me a estimation of completing a auto save functionality with unit testing in
// 🔹 Which cloud is used in your app development?

// 🔹 Security Features While Designing APIs

// Authentication & Authorization: Use OAuth2, JWT, or API keys.

// Input Validation: Sanitize and validate all inputs to prevent SQL injection and XSS.

// Rate Limiting / Throttling: Prevent abuse of APIs.

// HTTPS: Ensure encryption in transit.

// CORS Configuration: Restrict origins as needed.

// Error Handling: Avoid exposing sensitive info in errors.

// Data Encryption: Encrypt sensitive data at rest.

// Logging & Monitoring: Track suspicious requests.

// 🔹 What is API Throttling

// API throttling limits the number of API requests a client can make in a given time fr

// Prevents server overload and abuse.

// Example: Max 100 requests per minute per IP.

// 🔹 How to Improve Performance in React

// Use React.memo to prevent unnecessary re-renders.

// Use useCallback and useMemo to optimize expensive computations.

// Lazy load components with React.lazy + Suspense.

// Use Code splitting and dynamic imports.

// Optimize images and assets.

// Avoid anonymous functions inside JSX repeatedly.
```

```
// Use virtualization for large lists (react-window, react-virtualized).

// 🔹 What is Debouncing in React

// Debouncing ensures a function executes after a delay since the last trigger.

// Useful in search input filtering to reduce API calls.

// const handleInput = debounce((value) => console.log(value), 500);


// Only executes after 500ms of inactivity.

// 🔹 Auto-save Form Inputs in React

// Use useEffect with debouncing to send updates.

// useEffect(() => {
//   const handler = setTimeout(() => {
//     saveToBackend(formData);
//   }, 1000); // auto-save after 1 sec
//   return () => clearTimeout(handler);
// }, [formData]);


// Optionally, show a saving indicator to the user.

// 🔹 React Libraries for Charts/Graphs

// Recharts: Simple, lightweight, composable.

// Chart.js + react-chartjs-2: Popular, good for small-medium apps.

// Victory: Flexible and customizable.

// D3.js: Powerful but complex for large visualizations.

// 🔹 Informing Backend Developers About API Failures

// Use bug tracking tools like Jira, Trello.

// Send logs with request details, errors, and timestamps.

// Include API endpoint, request payload, and expected vs actual results.

// Optionally, create automated alerts via Slack or email.

// 🔹 Tools for Code Standards in React

// ESLint: Detects bad practices and potential bugs.

// Prettier: Auto-formats code for consistency.

```

```
5225    // Husky + lint-staged: Run pre-commit hooks for quality.
5226
5227    // 🔹 Unit Testing Tools in React
5228
5229    // Jest: Core testing framework.
5230
5231    // React Testing Library: Tests components from user perspective.
5232
5233    // Enzyme: Older library, used for shallow rendering.
5234
5235    // Vitest: Modern, lightweight alternative to Jest.
5236
5237    // 🔹 Handling API Test Case Scenarios
5238
5239    // Mock API responses using msw or jest.mock.
5240
5241    // Test loading, success, error states.
5242
5243    // Validate correct data rendering in UI.
5244
5245    // Simulate network errors for robustness.
5246
5247    // // Example
5248    // const { getByText } = render(<MyComponent />);
5249    // await waitFor(() => expect(getByText('Data Loaded')).toBeInTheDocument());
5250
5251    // 🔹 Estimation for Auto-Save Feature
5252    // Task Estimated Time
5253    // Form UI & state management   3-4 hours
5254    // Debounce logic + API integration 2-3 hours
5255    // Saving indicator & UX    1-2 hours
5256    // Unit tests (success/error/loading states)    3-4 hours
5257    // Total: ~9-13 hours (1-2 working days)
5258    // 🔹 Cloud Used in App Development
5259
5260    // AWS (S3, EC2, Lambda)
5261
5262    // Firebase (Auth, Firestore, Hosting)
5263
5264    // Azure or Google Cloud Platform for enterprise apps.
5265
5266    // ------------------------------------------------------------------------
5267
5268    // Senior React Interview Preparation – Day 1
5269
5270    // 🔹 Q1. How would you design a micro-frontend architecture with independent deploymen
5271    // 👉 A micro-frontend system breaks a large React application into smaller, independen
5272    // ✅ Use Module Federation (Webpack 5) for runtime integration
5273    // ✅ Define clear contracts between apps (via shared APIs or events)
5274    // ✅ Each team owns, builds, and deploys their micro-frontend separately
5275    // ✅ Isolate styling & dependencies to avoid conflicts
5276    // ✅ Deploy on a shared shell (container app) to integrate all parts
```

```
5277
5278    // 🔹 Q2. Compare different state management solutions in React (Redux, Context API, Zu
5279    // 👉 Trade-offs to know:
5280    // ✅ Redux – predictable, centralized, good for enterprise; but boilerplate-heavy
5281    // ✅ Context API – simple for theming & global states; not good for frequent updates (
5282    // ✅ Zustand – lightweight, scalable, less boilerplate, great DX
5283    // ✅ Recoil – fine-grained state updates + async selectors; not as widely adopted as R
5284
5285    // 🔹 Q3. How do you handle shared dependencies in a monorepo?
5286    // 👉 Monorepo dependency management strategies:
5287    // ✅ Use tools like Turborepo, Nx or Lerna
5288    // ✅ Hoist common dependencies to root to reduce duplication
5289    // ✅ Pin exact versions for critical libraries
5290    // ✅ Use package.json resolutions to manage conflicts
5291    // ✅ Automate builds with CI/CD pipelines to detect version mismatches
5292
5293    // ---------------------------------------------------------------------------
5294
5295    // Here the list of top programming hashtag#interview questions:
5296
5297    // 1. Reverse a String
5298    // 2. Check if a String is a Palindrome
5299    // 3. Remove Duplicates from a String
5300    // 4. Find the First Non-Repeating Character
5301    // 5. Count the Occurrences of Each Character
5302    // 6. Reverse Words in a Sentence
5303    // 7. Check if Two Strings are Anagrams
5304    // 8. Find the Longest Substring Without Repeating Characters
5305    // 9. Convert a String to an Integer (atoi Implementation)
5306    // 10. Compress a String (Run-Length Encoding)
5307    // 11. Find the Most Frequent Character
5308    // 12. Find All Substrings of a Given String
5309    // 13. Check if a String is a Rotation of Another String
5310    // 14. Remove All White Spaces from a String
5311    // 15. Check if a String is a Valid Shuffle of Two Strings
5312    // 16. Convert a String to Title Case
5313    // 17. Find the Longest Common Prefix
5314    // 18. Convert a String to a Character Array
5315    // 19. Replace Spaces with %20 (URL Encoding)
5316    // 20. Convert a Sentence into an Acronym
5317    // 21. Check if a String Contains Only Digits
5318    // 22. Find the Number of Words in a String
5319    // 23. Remove a Given Character from a String
5320    // 24. Find the Shortest Word in a String
5321    // 25. Find the Longest Palindromic Substring
5322
5323    // ---------------------------------------------------------------------------
5324
5325    // Most frequently asked 21 programs in L1 & L2 javascript interviews
5326
5327    // 1. Program to find longest word in a given sentence ?
5328    // 2. How to check whether a string is palindrome or not ?
5329    // 3. Write a program to remove duplicates from an array ?
```

```
// 3. Write a program to remove duplicates from an array ?
// 4. Program to find Reverse of a string without using built-in method ?
// 5. Find the max count of consecutive 1’s in an array ?
// 6. Find the factorial of given number ?
// 7. Given 2 arrays that are sorted [0,3,4,31] and [4,6,30]. Merge them and sort [0,3,4
// 8. Create a function which will accepts two arrays arr1 and arr2. The function should
// 9. Given two strings. Find if one string can be formed by rearranging the letters of
// 10. Write logic to get unique objects from below array ?
// I/P: [{name: "sai"},{name:"Nang"},{name: "sai"},{name:"Nang"},{name: "111111"}];
// O/P: [{name: "sai"},{name:"Nang"}{name: "111111"}
// 11. Write a JavaScript program to find the maximum number in an array.
// 12. Write a JavaScript function that takes an array of numbers and returns a new arra
// 13. Write a JavaScript function to check if a given number is prime.
// 14. Write a JavaScript program to find the largest element in a nested array.
// [[3, 4, 58], [709, 8, 9, [10, 11]], [111, 2]]
// 15. Write a JavaScript function that returns the Fibonacci sequence up to a given num
// 16. Given a string, write a javascript function to count the occurrences of each char
// 17. Write a javascript function that sorts an array of numbers in ascending order.
// 18. Write a javascript function that sorts an array of numbers in descending order.
// 19. Write a javascript function that reverses the order of words in a sentence withou
// 20. Implement a javascript function that flattens a nested array into a single-dimens
// 21. Write a function which converts string input into an object
// ("a.b.c", "someValue");
// {a: {b: {c: "someValue"}}}
// 22. Find 3rd least occurred number from an array


// ---------------------------------------------------------------------------

// Here are the Top 10 most common questions in javascript that can be asked in intervew
//  These are basic-level but frequently asked in interviews 👇

// 1️⃣ Call, Apply, and Bind
//  Difference between them + write a polyfill.

// 2️⃣ Flatten Array (No Array.flat)
//  Input : [1,2,3,[4,5,6,[7,8,[10,11]]],9]
//  Output: [1,2,3,4,5,6,7,8,10,11,9]

// 3️⃣ Inline 5 divs in a row without flex/margin/padding
//  (Hint: display: inline-block)

// 4️⃣ Find sum of numbers without for loop
//  Input: [1,2,3,4,5] — (Hint: reduce() or recursion)

// 5️⃣ Deep Copy vs Shallow Copy (How to achieve this)
//  const a = { ab: { cd: { ef: true } } };
//  const b = a; const c = { ...a };
//  console.log(a === b); console.log(a === c);
//  a.ab.cd.ef = false;
//  console.log(b.ab.cd.ef); console.log(c.ab.cd.ef);

// 6️⃣ Promise & Async/Await Output (More questions I will be posting later)
```

```
5382    //  async function chart(v){
5383    //  console.log("start", v);
5384    //  await console.log("middle", v);
5385    //  console.log("end", v);
5386    //  }
5387    //  chart("first");
5388    //  chart("second");
5389
5390    // 7️⃣ Find first repeating character
5391    //  Example: "success" → c
5392
5393    // 8️⃣ Implement a Stopwatch
5394    //  Start, Stop, Reset + live display.
5395
5396    // 9️⃣ Build a To-Do List (Vanilla JS or React)
5397    //  Follow-up: How would you optimize re-renders?
5398
5399    // 🔟 Currying Function for Infinite Sum.
5400    //  sum(10)(20)(30)() Output => 60
5401    //  sum(10)(20)(30)(40)(50)(60)() Output => 210
5402
5403    // ---------------------------------------------------------------------------
5404
5405    // 🗝 Core JavaScript
5406    // 1). Event loop order: Promise.then vs setTimeout(0) vs
5407    // requestAnimationFrame - which fires first and why?
5408    // 2). this binding: arrow vs function vs class methods (bugs
5409    // you've probably hit).
5410    // 3). Deep copy: why JSON.parse/stringify breaks (Dates
5411    // Maps, Sets, functions) + safe options.
5412    // 4). Debounce vs throttle -- implement your own mini
5413    // version.
5414    // 5). Coercion gotchas: [] + {} and {} + [] - explain the output.
5415
5416    // ⚛️ React
5417    // 1). What actually triggers a re-render? (5 causes)
5418    // 2). useEffect dependencies & stale closures -- when to
5419    // refactor.
5420    // 3). When not to use useMemo/useCallback.
5421    // 4). State colocation vs Context vs external store (Redux/
5422    // Zustand).
5423    // 5). Suspense + Error Boundaries - modern fetch flows.
5424
5425    // 🎨 CSS/ Layout
5426    // 1). Flex vs Grid - picking the right primitive.
5427    // 2). Stacking context: why your z-index: 9999 still loses
5428    // 3). Container queries in responsive design.
5429    // 4). position: sticky -- why it sometimes doesn't work.
5430
5431    // ⚡ Performance
5432    // 1). LCP / INP / CLS - what "good" looks like & how to
5433    // improve.
```

```
// 2). Code-splitting strategy: routes, vendors, preloading
// 3). Image pipeline: formats, loading, content-visibility

// ♿ Accessibility (A11y)
// 1). Form errors that announce correctly (ARIA, live
// regions).
// 2). Keyboard navigation & roving tabindex
// 3). Accessible dialogs: focus trap, inert background,
// aria-modal.

// 🧪 Testing & Tooling,
// 1). Unit vs integration vs E2E -- with examples.
// 2). Mocking network calls (MSW or similar)
// 3). Bundle analysis: fixing a runaway dependency.

// 🏗️ Frontend System Design
// 1). Notifications dropdown: pagination, real-time
// optimistic UI.
// 2). Feature flags on the client: config delivery, guardrails
// fallbacks.

// 💡Bonus challenge (15 min):
// Your search box feels sluggish on every keystroke. How
// would you diagnose and ship a fix?
```